

Lycée Guillaume Kroll

BTS Cloud Computing — B1CLC, Promotion 2024

Cloud Project (CLOPR2)



Recap: AI-Powered Meeting & Lecture Summarizer

Technical Project Report

DURAES VALADARES David

DurDa593

March 2026

Declaration of Honour

I the undersigned solemnly declare that the project report Cloud Project (CLOPR2) is based on my own work carried out during the BTS Cloud Computing programme at Lycée Guillaume Kroll.

I assert the statements made and conclusions drawn are an outcome of my research work. I further certify that:

- The work contained in this report is original and has been done by myself.
- I have not copied from any source without proper attribution.
- AI tools (Claude Code by Anthropic) were used as a development assistant. All AI usage is disclosed in [Section 7](#).

DURAES VALADARES David

Date: 24th March 2026

Table of Contents

Declaration of Honour	2
1. Project Description	6
1.1 Overview	6
1.2 Problem Statement	6
1.3 Goals	6
1.4 Implemented Features	7
1.5 Technology Stack	7
2. Architecture Diagram	8
2.1 Data Flow	9
2.2 Network Layout	10
2.3 Project Folder Structure	12
2.4 Azure Resources	13
3. Mandatory Technologies	15
3.1 Azure IAM (Azure AD / Entra ID)	15
Where	15
Why	15
How	16
3.2 Proxmox On-Premise	17
Where	17
Why	17
How	18
3.3 Docker Containerization	19
Where	19
Why	19
How	19
3.4 Kubernetes Orchestration (AKS)	20
Where	20
Why	20

How	20
3.5 Terraform (Infrastructure as Code)	21
Where	21
Why	21
How	21
3.6 Ansible (Configuration Management)	24
Where	24
Why	24
How	24
3.7 Monitoring (Prometheus + Grafana)	25
Where	25
Why	25
How	25
3.8 Naming Conventions & Resource Tagging	30
Naming Convention	30
Resource Tagging	31
Azure Policies	31
3.9 Operations Dashboard	32
Where	32
Why	32
How	32
4. Major Challenges	33
4.1 AKS Resource Constraints — Fitting a Full Stack into 16 GB of RAM	33
The Problem	33
Solution 1: Whisper OOMKill — Raise Memory Limit to 1 Gi	33
Solution 2: Soft Anti-Affinity for High Availability	34
Solution 3: Conservative HPA with Stabilization Windows	34
Quota Constraints — Unresolved	35
Result	35
Lessons Learned	35
4.2 Google Gemini Free Tier — Building a Reliable AI Pipeline on Unpredictable Infrastructure	35

The Problem	35
Solution 1: Sliding Window Rate Limiter with Cooldown.....	36
Solution 2: String-Matching Error Detection.....	36
Solution 3: JSON Fallback Parser	37
Solution 4: Fake System Role for Chat Context.....	37
Solution 5: Stale Recording Recovery.....	37
Result	38
Lessons Learned	40
5. Security Concept.....	40
5.1 Cloud Security (Azure)	41
5.2 On-Premise Security (Proxmox)	42
5.3 Application Security.....	42
5.4 Network Security	43
5.5 EU Compliance Note.....	43
6. Cost Usage	44
6.1 Azure Resource Costs	44
6.2 Two-Subscription Strategy	45
6.3 Cost Optimization Strategies.....	46
6.4 Actual Spend.....	46
7. Usage of AI.....	46
7.1 How AI Was Used	46
7.2 AI Prompts	47
8. Personal Conclusion	47
8.1 Project Reflection	47
8.2 Project Management Reflection	48
9. Planned Improvements	48
Appendix A: AI Prompts	49
Table of Figures	50

1. Project Description

1.1 Overview

Recap is a web application that turns meetings and lectures into structured notes. Users can upload audio/video files, record directly in the browser, paste text, or drop in a YouTube link. The app transcribes the content with Whisper, then sends it to Google Gemini for a summary, key points, and action items. There is also an AI chat per recording for follow-up questions.

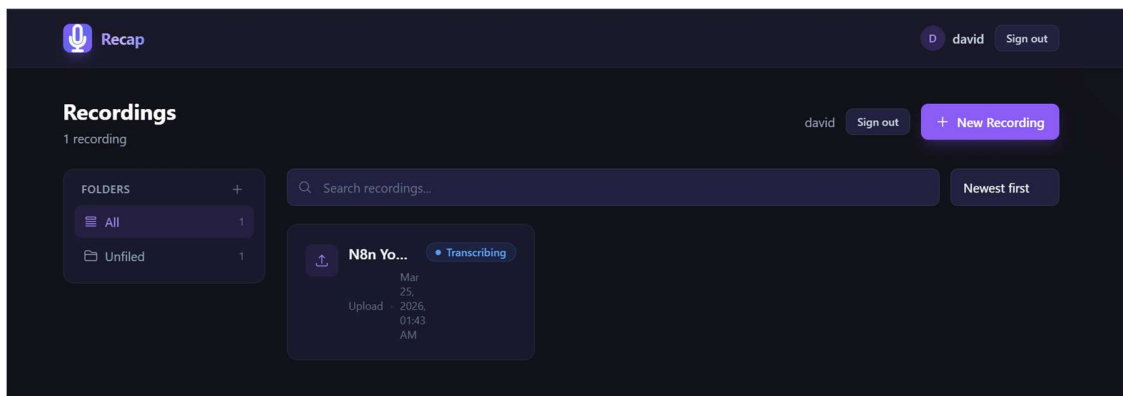


Figure 1 — Recap Dashboard — Recordings List

1.2 Problem Statement

Most people do not take good notes during meetings or lectures, and reviewing a 60-minute recording just to find one key decision is painful. I wanted to build something that does the work for you: give it audio, and get back a summary, key points, and action items you can actually use.

1.3 Goals

- Run a multi-user web app on hybrid infrastructure (Azure + Proxmox on-premise)
- Transcribe audio/video with self-hosted Whisper (no paid API)
- Summarize recordings and extract action items via Google Gemini
- Let users ask follow-up questions about each recording through AI chat
- Automate everything with Terraform and Ansible so the whole stack is reproducible
- Monitor cloud and on-premise infrastructure with Prometheus and Grafana
- Apply security at every layer (network, application, data)
- Keep costs as close to zero as possible using free-tier resources

1.4 Implemented Features

- Login with Microsoft (Azure AD multi-tenant) or email/password (bcrypt + JWT)
- Four input methods: file upload, in-browser mic recording, YouTube link, text paste
- Transcription via self-hosted Whisper (base model, CPU-only on AKS)
- AI summaries, key points, and action items via Gemini 2.5 Flash-Lite (free tier)
- Per-recording AI chat with conversation history
- User dashboard with folders, search, and sorting
- Operations dashboard: 4-page web app for infrastructure management, one-click deployment, real-time logs
- Real-time processing status (WebSocket polling)
- Fully automated deployment: Terraform + Ansible, empty subscription to live app in ~16 minutes
- Monitoring: 6 Grafana dashboards, 32 alert rules, 14 scrape jobs (27 targets)
- Disaster recovery: streaming replication, daily backups, automatic failover via repmgr + keepalived
- Zero-trust networking: 20 NetworkPolicies across 2 namespaces
- PodDisruptionBudgets for backend, whisper, and prometheus
- Architecture diagrams generated from Mermaid definitions

1.5 Technology Stack

Layer	Technology	Runs On
Frontend	Next.js 14 (React 18), Tailwind CSS	AKS (Azure)
Backend	Python FastAPI, SQLAlchemy 2.0	AKS (Azure)
Transcription	OpenAI Whisper (self-hosted, base model, CPU-only)	AKS (Azure)
AI Summarization & Chat	Google Gemini 2.5 Flash-Lite (free tier)	External API
Database	PostgreSQL 16 (primary + hot standby)	Proxmox (on-prem)
Authentication	Azure AD (Entra ID) multi-tenant + local bcrypt/JWT	Azure + AKS
File Storage	Azure Blob Storage (Standard LRS, Hot)	Azure
Secrets	Azure Key Vault (CSI driver + workload identity)	Azure
Container Registry	Azure Container Registry (Basic)	Azure
Monitoring	Prometheus + Grafana + Blackbox Exporter	AKS (monitoring ns)
IaC	Terraform (Azure + Proxmox) + Ansible (8 playbooks, 7 roles)	All environments
VPN	WireGuard (B2ats_v2 VM)	Azure ↔ Proxmox

Figure 2 — Technology Stack

2. Architecture Diagram

The system is split across two environments (Figure 3). Azure (swedencentral) runs the AKS cluster with two D2s_v4 nodes for the application and monitoring. The school's Proxmox hypervisor hosts the PostgreSQL database on-premise because a managed database would eat through the student credit too fast.

Traffic comes in through NGINX Ingress with a public IP and TLS from Let's Encrypt (cert-manager). The ingress routes `/api/*` to the FastAPI backend and everything else to the Next.js frontend. From there the backend talks to the Whisper worker for transcription, Gemini for summarization, Blob Storage for file uploads, and PostgreSQL for everything else. A WireGuard tunnel (UDP 51820) on a small B2ats_v2 VM connects Azure to Proxmox, with DNAT forwarding database traffic (5432), node metrics (9100), and PostgreSQL metrics (9187) — all three to the keepalived VIP.

A separate monitoring namespace runs Prometheus (14 scrape jobs, 32 alert rules), Grafana (6 dashboards), and a Blackbox Exporter for endpoint probes. Auth goes through Azure AD or local email/password. Container images live in ACR, secrets in Key Vault (CSI driver), and Azure Policy handles governance.

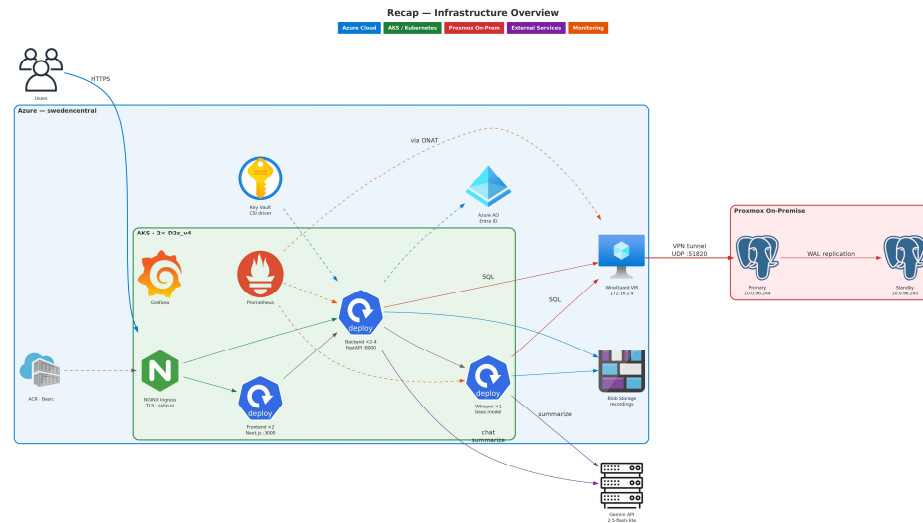


Figure 3 — Infrastructure Architecture Diagram

2.1 Data Flow

A recording goes through several stages from input to output (Figure 4). Users can upload a file (MP3/MP4/WAV, max 100 MB), record with their browser mic, paste a YouTube URL, or type/paste text. The frontend validates the input and sends it to the backend API.

The backend saves the file to Azure Blob Storage and creates a database record with status “pending”. The Whisper worker picks it up on its next poll cycle, downloads the file, and transcribes it (Whisper base model, CPU-only, needs about 1 GB RAM). After transcription the status flips to “summarizing”, and the worker sends the transcript to Gemini (free tier: 14 RPM, 1000 RPD). Gemini returns a summary, key points, and action items. Everything gets saved to PostgreSQL and the status becomes “completed”.

Meanwhile the frontend polls at increasing intervals (3s, 5s, 10s, 15s) and renders the results once they are ready. After that, users can ask follow-up questions through the AI chat, which also calls Gemini with the transcript as context.

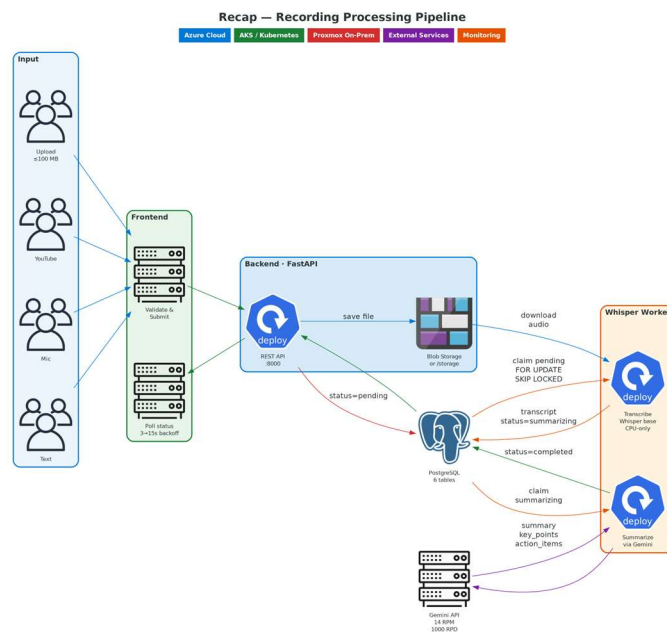


Figure 4 — Data Flow Diagram

2.2 Network Layout

Network	CIDR / Address	Purpose
Azure VNet	172.16.0.0/16	Cloud networking (vnet-b2clc-durda-recap)
AKS Subnet	snet-aks (172.16.1.0/24)	Kubernetes pods and services
WG Subnet	snet-b2clc-durda-wg (172.16.2.0/24)	WireGuard VM
GatewaySubnet	GatewaySubnet (172.16.255.0/27)	Reserved for future VPN Gateway
WireGuard VM	172.16.2.4	VPN gateway + DNAT to Proxmox
WG Tunnel	10.99.0.0/24	Point-to-point encrypted tunnel
Proxmox Primary	10.0.96.244	PostgreSQL primary (VM ID 593)
Proxmox Standby	10.0.96.245	PostgreSQL hot standby (VM ID 594)
UDR Route	10.0.0.0/16 via 172.16.2.4	AKS → Proxmox routing

Figure 5 — Network Layout

The Azure VNet (172.16.0.0/16) is divided into three subnets (Figure 6): snet-aks for Kubernetes pods, a dedicated subnet for the WireGuard VM (172.16.2.4), and a GatewaySubnet reserved for a potential VPN Gateway migration later. A UDR sends all 10.0.0.0/16 traffic through the WireGuard VM.

The WireGuard VM opens an encrypted tunnel (UDP 51820) to the Proxmox LAN and uses DNAT to forward database connections (5432) to the keepalived VIP (10.0.96.250) and monitoring scrapes (9100, 9187) to the primary VM. I had to use DNAT because direct UDR routing from AKS pods to Proxmox IPs simply does not work — I spent a full day debugging this before switching to explicit forwarding on the WireGuard VM.

On the Proxmox side, the primary (10.0.96.244) streams WAL to the standby (10.0.96.245). Keepalived manages the VIP so that if the primary goes down, repmgr promotes the standby and the VIP floats over automatically. AKS does not need any reconfiguration.

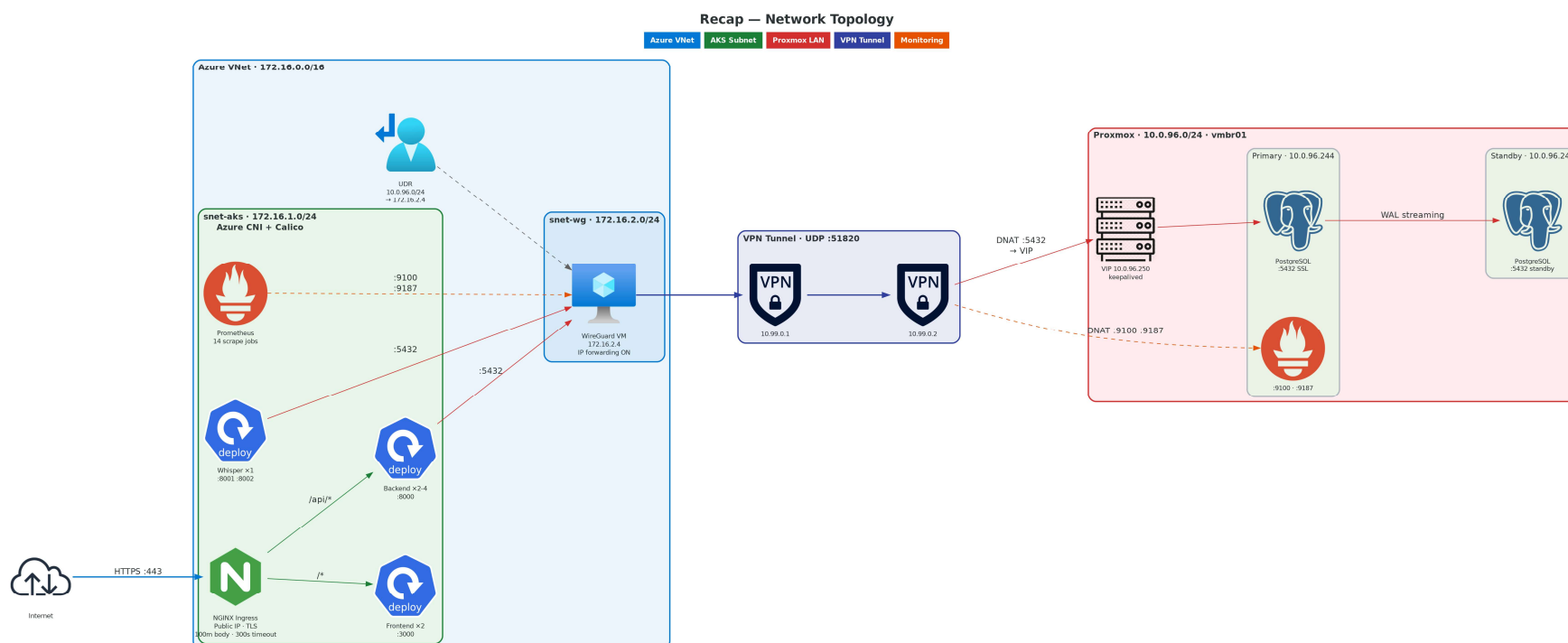


Figure 6 — Network Topology Diagram

2.3 Project Folder Structure

```
recap/
  frontend/      # Next.js 14 app (src/app/, components/, lib/)
  backend/       # FastAPI (app/main.py, routers/, services/, models/, core/)
  whisper-worker/ # Whisper inference worker (worker.py)
  dashboard/     # Operations dashboard (Next.js + FastAPI)
  k8s/
    base/        # namespace, configmap, secret-provider, storage
    apps/        # 3 deployments + 3 services
    ingress/     # ingress + cert-manager-issuer
    monitoring/  # prometheus, grafana, blackbox (9 files)
    policies/    # network-policies, hpa, pdb
  terraform/
    azure/       # 13 .tf files + cloud-init template
    proxmox/     # 2 VM definitions (primary + standby)
  ansible/
    playbooks/   # 8 playbooks
    roles/       # 7 roles (postgresql, common, wireguard,
                  #   node_exporter, postgres_exporter, repmgr, keepalived)
    inventory/   # proxmox.yml (db_primary, db_standby groups)
  scripts/
    deploy/      # deploy-full.sh, deploy-aks.ps1, dr-failover.sh
    azure/       # 5 audit/discovery scripts
    ops/         # start/stop-working, check-monitoring/network
    helpers/     # terraform-import.sh, generate-report.py
    archive/     # 14 deprecated one-time scripts
  docs/
    architecture/ # proposals, architecture refs
    diagrams/     # 10 PNG + 6 SVG diagrams + render.py
    reports/      # technical report, drafts
    planning/     # user stories, ClickUp, week plan
    guides/       # deployment guides, guidance.txt
  .github/workflows/ # CI/CD
  docker-compose.yml # Local development (4 services)
  Makefile        # 40+ targets
```

Figure 7 — Project Folder Structure

2.4 Azure Resources

The production deployment runs on the primary subscription (Azure for Students durda, ID 468097ec-c4ca-43e9-bd10-b5740c6acc68). A second subscription (durda1, 266c0b0d-0352-4f21-b1ba-440cab57ccf3) was used for Terraform testing — see [Section 6.2](#). The primary has 21 resources across 3 resource groups. I manage rg-b2clc-durda-recap directly; the MC_ resource group is created and managed by AKS automatically.

Resource	Type	Details
aks-b2clc-durda-recap	AKS Cluster	K8s 1.33.7, 2x D2s_v4, free control plane
vm-b2clc-durda-wg	Virtual Machine	B2ats_v2, WireGuard VPN gateway
acrb2clcdurdarecap	Container Registry	Basic SKU, 3 repos (backend/frontend/whisper)
stb2clcdurdarecap	Storage Account	Standard LRS Hot, HTTPS-only, TLS 1.2
kv-b2clc-durda-recap	Key Vault	8 secrets, CSI driver integration
vnet-b2clc-durda-recap	Virtual Network	172.16.0.0/16, 3 subnets
id-b2clc-durda-recap	Managed Identity	Workload identity for AKS → Key Vault
rt-b2clc-durda-aks	Route Table	UDR: 10.0.0.0/16 → 172.16.2.4
nsg-b2clc-durda-recap	NSG	Allow HTTP/HTTPS inbound
nsg-b2clc-durda-wg	NSG	Allow WireGuard (51820), SSH, PostgreSQL
pip-b2clc-durda-wg	Public IP	4.223.131.219 (WG VM, static)

Figure 8 — Azure Resources (rg-b2clc-durda-recap)

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

durda1@bts.lgk.lu

LYCÉE GUILLAUME KROLL - BTS (...)

Home > Azure for Students durda1 | Resource groups

rg-b2clc-durda-recap

Resource group

How to manage changes with deployment tools?

Generate Bicep code to duplicate this resource group.

Export resource groups using Bicep or Terraform

Search

Create

Manage view

Delete resource group

Refresh

Export to CSV

Open query

Assign tags

Move

Delete

Export template

Group by none

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

Subscription (move): Azure for Students durda1

Subscriptions ID: 266c0b0d-0352-4f21-b1ba-440cab57cf3

Tags (edit): Environment: prod, Owner: DURDA, Show more (2)

Deployments: No deployments

Location: Sweden Central

JSON View

Resources

Recommendations

Filter for any field... Type equals all Location equals all Add filter

Name ↑	Type	Location
acrb2clcdurdarecap2	Container registry	Sweden Central
aks-b2clc-durda-recap	Kubernetes service	Sweden Central
id-b2clc-durda-recap	Managed Identity	Sweden Central
kv-b2clc-durda-recap2	Key vault	Sweden Central
nic-b2clc-durda-wg	Network Interface	Sweden Central
nsg-b2clc-durda-recap	Network security group	Sweden Central
nsg-b2clc-durda-wg	Network security group	Sweden Central
pip-b2clc-durda-wg	Public IP address	Sweden Central
rt-b2clc-durda-aks	Route table	Sweden Central
stb2clcdurdarecap2	Storage account	Sweden Central
vm-b2clc-durda-wg	Virtual machine	Sweden Central
vm-b2clc-durda-wg_disk1_3c5a300a7b864d7c99c9738556107cc6	Disk	Sweden Central
vnet-b2clc-durda-recap	Virtual network	Sweden Central

Figure 9 — Azure Portal — Resource Group Overview

3. Mandatory Technologies

3.1 Azure IAM (Azure AD / Entra ID)

Where

I registered an app called "Recap" in Azure AD (Client ID: 0efcb593-e873-469b-b126-cd34ce102ebb). The audience is set to "AzureADandPersonalMicrosoftAccount", which means any Microsoft account can sign in — school accounts, work accounts, and personal Outlook/Hotmail accounts. On the frontend, MSAL.js handles the OAuth login popup. On the backend, PyJWT validates the token using Microsoft's public JWKS keys to make sure it was not forged.

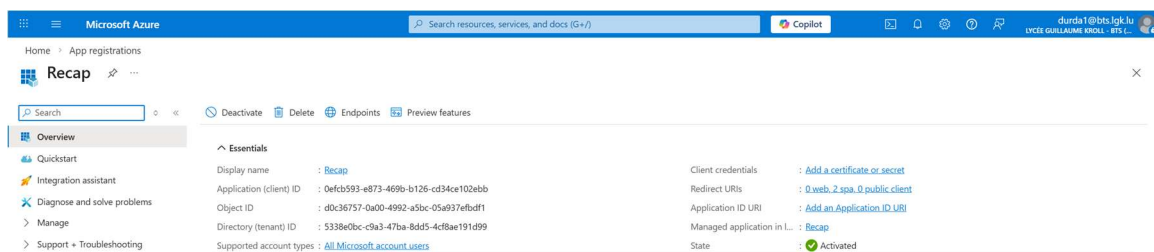


Figure 10 — Azure AD App Registration — Recap

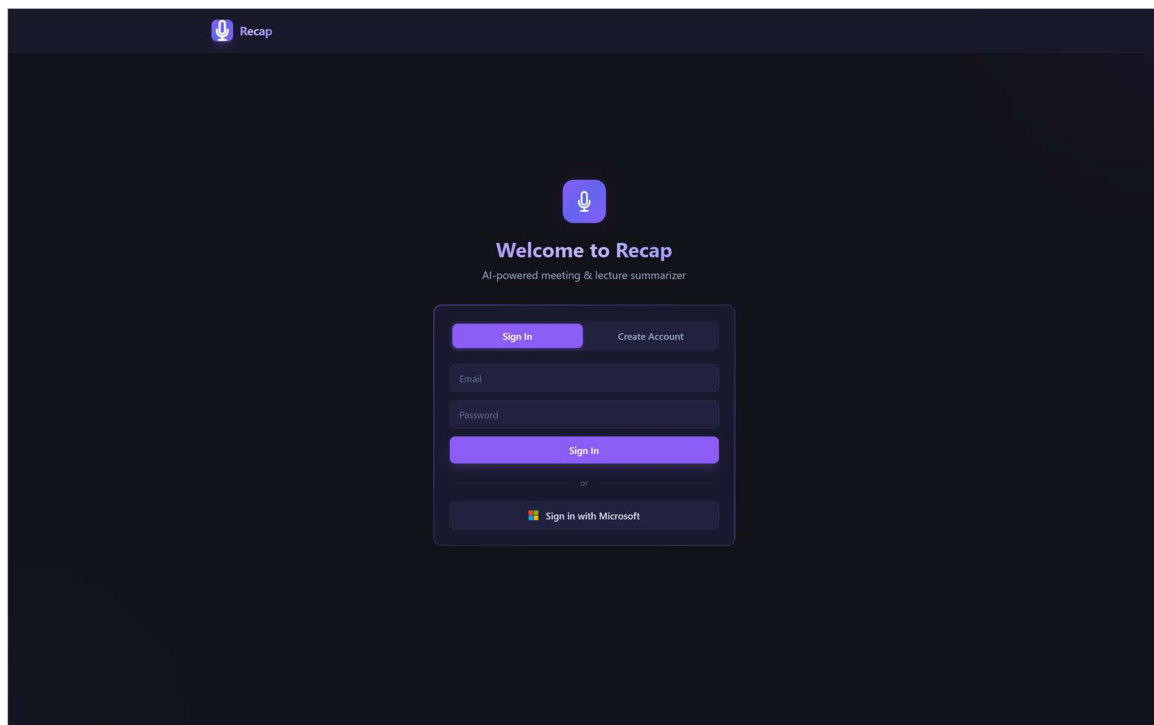


Figure 11 — Login Page — Microsoft + Email/Password Tabs

Why

Azure IAM is a project requirement. I chose Azure AD because it lets any Microsoft account sign in without extra configuration.

How

The frontend uses `@azure/msal-browser` for OAuth 2.0 with PKCE. I also added a local email/password login (bcrypt + JWT) because Azure AD B2C was discontinued in May 2025 and Entra External ID is blocked by the subscription's `sys.regionrestriction` policy. So users can log in either way.

```
# frontend/src/lib/msal-config.ts - MSAL configuration
export const msalConfig: Configuration = {
  auth: {
    clientId,
    authority: "https://login.microsoftonline.com/common", // multi-tenant
    redirectUri: typeof window !== "undefined" ? window.location.origin :
    "http://localhost:3000",
  },
  cache: { cacheLocation: "sessionStorage", storeAuthStateInCookie: false },
};
```

The backend accepts both token types. It tries to decode a local JWT first; if that fails, it falls through to Azure AD JWKS validation:

```
# backend/app/core/dependencies.py - dual-auth (lines 27-74)
async def get_current_user(credentials, db) -> User:
    # Try local JWT first (from /api/auth/login)
    try:
        payload = pyjwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        user_id = payload.get("sub")
        if user_id:
            user = await db.execute(select(User).where(User.id == uuid.UUID(user_id)))
            if user: return user
    except pyjwt.InvalidTokenError:
        pass # Not a local JWT - fall through to Azure AD

    # Try Azure AD token (RS256 via JWKS)
    if settings.auth_enabled:
        claims = validate_token(token) # -> backend/app/core/auth.py
```

For Azure AD tokens, I use a cached JWKS client (1-hour TTL) and validate the issuer pattern so any Azure tenant works:

```
# backend/app/core/auth.py - Azure AD JWKS validation (lines 39-60)
ISSUER_PATTERN = re.compile(r"^https://login\.microsoftonline\.com/[0-9a-f-]+/v2\.0$")

def validate_token(token: str) -> dict:
    signing_key = _get_jwks_client().get_signing_key_from_jwt(token)
    claims = jwt.decode(token, signing_key.key, algorithms=["RS256"],
                        audience=settings.azure_client_id)
    if not ISSUER_PATTERN.match(claims.get("iss", "")):
        raise jwt.InvalidTokenError("Invalid issuer")
    return claims
```

Figure 12 shows both auth paths side by side. On the left, MSAL.js starts an OAuth 2.0 PKCE flow with Azure AD, gets an access token back, and the backend validates it via JWKS (RS256). On the right, users register with email/password (bcrypt hashed, stored in PostgreSQL) and log in to get a HS256 JWT (24-hour expiry). Both paths end up at the same `get_current_user` dependency, so the rest of the API does not care which method was used.

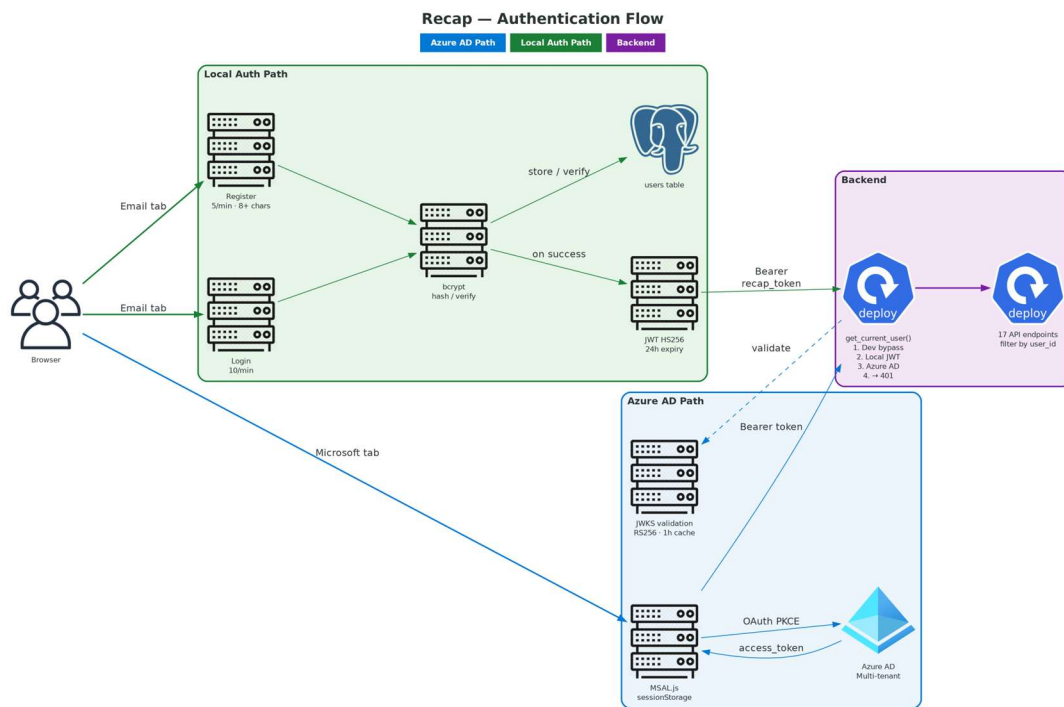


Figure 12 — Authentication Flow Diagram (Azure AD + Local Auth)

3.2 Proxmox On-Premise

Where

Two Ubuntu 22.04 VMs on the school's Proxmox hypervisor: primary (10.0.96.244, VM ID 593) and hot standby (10.0.96.245, VM ID 594). Both provisioned by Terraform (bpg/proxmox provider).

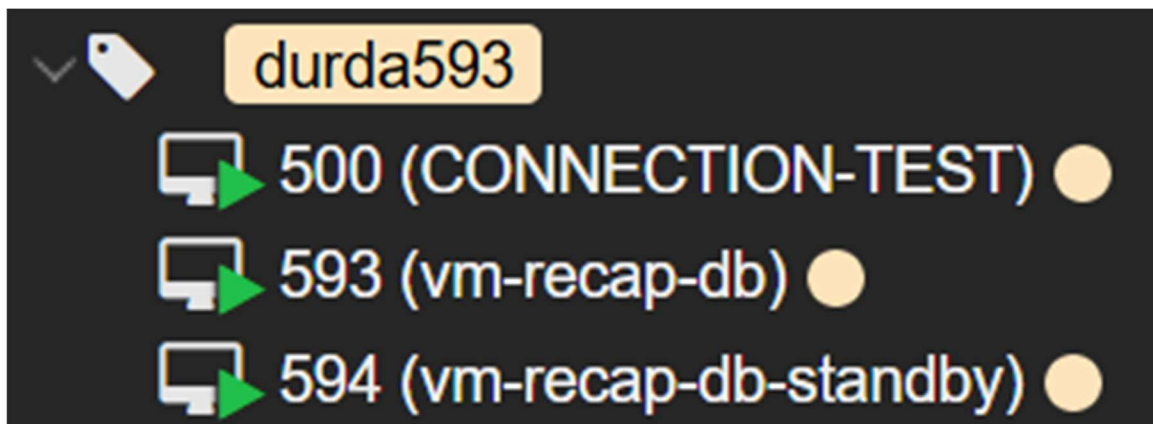


Figure 13 — Proxmox — Database VMs (Primary + Standby)

Why

Hybrid cloud is a project requirement, and running the database on-premise saves real money. Azure Database for PostgreSQL starts around \$30/month; on Proxmox it costs nothing.

How

I install PostgreSQL 16 via Ansible, bootstrap the standby with pg_basebackup, and set up automatic failover using repmgr + keepalived (VIP 10.0.96.250):

```
# ansible/playbooks/setup-db-replication.yml – standby bootstrap (lines 45-58)
- name: Run pg_basebackup from primary
  become_user: postgres
  ansible.builtin.command:
    cmd: >-
      pg_basebackup
      -h {{ pg_primary_host }}
      -U {{ replication_user }}
      -D {{ pg_data_dir }}
      -Fp -Xs -P -R          # -R creates standby.signal + primary_conninfo
      -S standby_slot
  when: not standby_signal.stat.exists
```

Disaster recovery has three layers (Figure 14). During normal operation the primary handles all writes and streams WAL to the standby in real time. A pg_dump runs at 3:00 AM daily (7-day retention), gets rsync'd to the standby at 3:30 AM, and every Sunday at 5:00 AM a cron job restores the backup to a temporary database, checks the tables, and drops it — just to make sure the backup is actually usable.

If the primary goes down, repmgrd (which monitors it continuously) promotes the standby after 3 failed reconnect attempts. Keepalived moves the VIP (10.0.96.250) to the new primary, and because the WireGuard VM's DNAT points to the VIP rather than a specific IP, AKS does not notice anything changed. No pod restarts, no config changes. Prometheus watches the whole thing with 4 alerts: BackupTooOld, BackupMissing, ReplicationFailed, and BackupVerificationFailed.

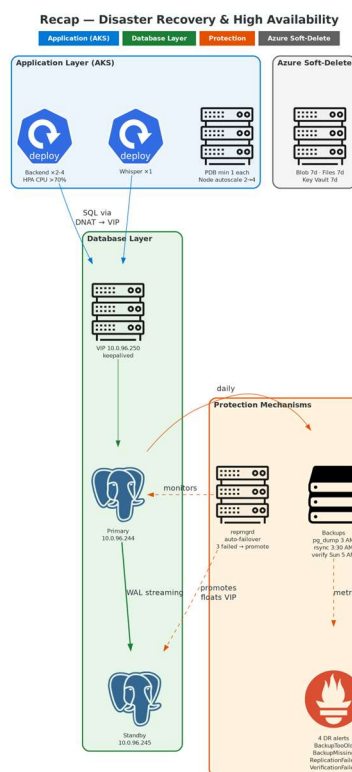


Figure 14 — Disaster Recovery & Automatic Failover Flow

The database has 6 tables (Figure 15). The main table is recordings, which belongs to a user and optionally to a folder. Each recording has one transcript, one summary, and many chat_messages. The users table handles both Azure AD users (azure_ad_id column) and local users (password_hash). Summaries store key_points and action_items as JSON arrays so the frontend can render them as lists.

3.3 Docker Containerization

Where

Three Docker images pushed to ACR (acrb2clcdurdarecap.azurecr.io): recap-frontend, recap-backend, recap-whisper. All tagged :latest.

Why

Docker containerization is mandatory. Each microservice has its own image with pinned dependencies.

How

All three containers run as a non-root user (UID 1000). The tricky part was the whisper-worker Dockerfile: by default, pip pulls the full PyTorch with CUDA support (3.3 GB). I force CPU-only:

```
# whisper-worker/Dockerfile - CPU-only torch + non-root user
COPY requirements.txt .
RUN pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu \
    && pip install --no-cache-dir --no-build-isolation -r requirements.txt \
    && pip uninstall -y triton \
    && rm -rf /root/.cache/pip /tmp/*

COPY worker.py .

RUN adduser --system --uid 1000 --no-create-home recap
RUN mkdir -p /home/recap/.cache && chown -R recap /home/recap
USER recap
ENV HOME=/home/recap
```

I build images locally because ACR Tasks are not available on Azure for Students, then push with docker push.

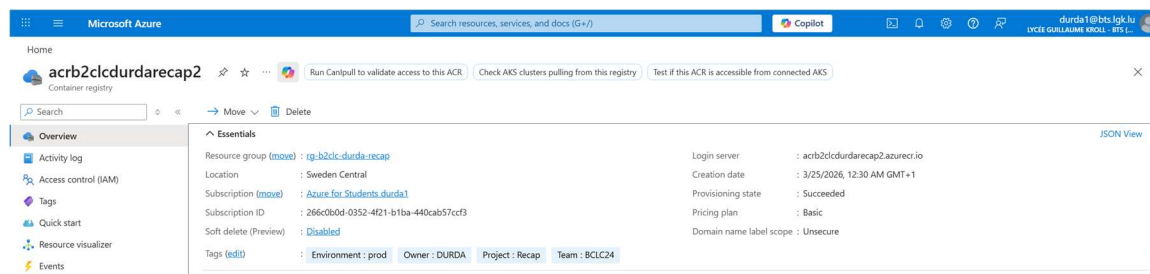


Figure 16 — Azure Container Registry — 3 Docker Images

3.4 Kubernetes Orchestration (AKS)

Where

Azure Kubernetes Service — cluster aks-b2clc-durda-recap, 2-node pool (Standard_D2s_v4), Kubernetes 1.33.7, swedencentral. Free-tier control plane.

Why

Kubernetes orchestration is mandatory. AKS offers a free control plane, keeping compute costs limited to the worker nodes.

How

The cluster runs 5 application pods (recap-app namespace) and monitoring pods (monitoring namespace). NGINX Ingress handles TLS termination with Let's Encrypt certs from cert-manager. The backend has an HPA that scales from 2 to 4 replicas when CPU goes above 70%. Secrets come from Key Vault through the CSI driver with workload identity, so no secrets are hardcoded anywhere. Everything is defined in 25 manifest files across 5 subdirectories. I also added PodDisruptionBudgets so that kubectl drain cannot kill all replicas at once.

Pod resource allocation:

Pod	Replicas	CPU Req/Limit	Memory Req/Limit	Notes
Frontend	2	100m / 200m	128Mi / 256Mi	Soft anti-affinity
Backend	2	150m / 300m	192Mi / 384Mi	HPA: 2–4 pods
Whisper Worker	1	250m / 500m	512Mi / 1Gi	base model, CPU-only

Figure 17 — Pod Resource Allocation

```
david@david:~/project/Project_start$ kubectl get pods -n recap-app
NAME                                READY   STATUS    RESTARTS   AGE
backend-798d7d97cc-jhg88            1/1     Running   0           9m51s
backend-798d7d97cc-zsrln            1/1     Running   0           10m
frontend-c8c7467dd-k4drw            1/1     Running   0           77m
frontend-c8c7467dd-lr8lf            1/1     Running   0           77m
whisper-worker-85c7698944-ztdsh     1/1     Running   0           42m
david@david:~/project/Project_start$
```

Figure 18 — Running Pods in recap-app Namespace

K8s manifest organization (25 files across 5 subdirectories):

Directory	Files	Contents
k8s/base/	5	namespace, configmap, secret-provider, secret template, storage PVC
k8s/apps/	6	3 deployments + 3 services (backend, frontend, whisper)
k8s/ingress/	2	ingress + cert-manager ClusterIssuer
k8s/monitoring/	9	prometheus, grafana, blackbox-exporter, configs, RBAC, ingress
k8s/policies/	3	20 NetworkPolicies, HPA (backend 2–4), 3 PDBs

Figure 19 — Kubernetes Manifest Organization

3.5 Terraform (Infrastructure as Code)

Where

17 Terraform files split into two directories: terraform/azure/ (13 .tf files + cloud-init template) and terraform/proxmox/ (5 files). Azure provider: azurerm ~> 3.116. Proxmox provider: bpg/proxmox.

Why

IaC is a project requirement. Having everything in Terraform means I can destroy and recreate the full infrastructure from an empty subscription in about 16 minutes (tested 2026-03-24). Splitting into azure/ and proxmox/ lets me plan/apply each environment independently.

How

The Azure configuration creates ~46 resources total. Here is the AKS cluster definition with the important choices: free tier, node autoscaling 2–4, Calico for NetworkPolicies, workload identity:

```
# terraform/azure/aks.tf – AKS cluster (lines 8-40)
resource "azurerm_kubernetes_cluster" "recap" {
  name           = "aks-${local.name_suffix}"
  sku_tier       = "Free"
  tags           = local.common_tags

  default_node_pool {
    vm_size          = "Standard_D2s_v4"
    vnet_subnet_id   = azurerm_subnet.aks.id
    auto_scaling_enabled = true
    min_count        = 2
    max_count        = 4
  }

  network_profile {
    network_plugin = "azure"
    network_policy = "calico" # required for K8s NetworkPolicies
  }

  oidc_issuer_enabled = true
  workload_identity_enabled = true # for Key Vault CSI driver
  key_vault_secrets_provider { secret_rotation_enabled = false }
}
```

I also added two terraform_data helpers to handle Key Vault soft-delete purge and orphaned budget cleanup, because both caused terraform apply to fail on the second run without them.

Directory / File	Resources
terraform/azure/resource-group.tf	rg-b2clc-durda-recap (swedencentral)
terraform/azure/aks.tf	AKS cluster, node pool (2x D2s_v4), OIDC, workload identity
terraform/azure/networking.tf	VNet, 3 subnets, UDR, NSGs
terraform/azure/storage.tf	Storage account, blob container, lifecycle policy (Cool after 30d)
terraform/azure/keyvault.tf	Key Vault, RBAC role assignments, 8 secrets
terraform/azure/acr.tf	Container Registry (Basic SKU)
terraform/azure/vpn.tf	WireGuard VM (B2ats_v2), cloud-init, public IP, NSG

terraform/azure/policies.tf	7 Azure Policy assignments
terraform/azure/cost-management.tf	Consumption budget (\$50/month alert)
terraform/proxmox/main.tf	2 VMs (primary + standby) via bpg/proxmox

Figure 20 — Terraform Configuration Files

```
# terraform_data.cleanup_keyvault will be created
+ resource "terraform_data" "cleanup_keyvault" {
  + id          = (known after apply)
  + triggers_replace = [
    + (known after apply),
  ]
}
```

Plan: 46 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ aad_client_id          = (known after apply)
+ aad_object_id          = (known after apply)
+ acr_admin_password     = (sensitive value)
+ acr_admin_username     = (known after apply)
+ acr_login_server       = (known after apply)
+ aks_cluster_name       = "aks-b2clc-durda-recap"
+ aks_kube_config        = (sensitive value)
+ aks_oidc_issuer_url    = (known after apply)
+ db_password            = (sensitive value)
+ key_vault_name         = "kv-b2clc-durda-recap2"
+ key_vault_uri          = (known after apply)
+ managed_identity_client_id = (known after apply)
+ resource_group_name     = "rg-b2clc-durda-recap"
+ storage_account_name    = "stb2clcdurdarecap2"
+ storage_connection_string = (sensitive value)
+ wireguard_private_ip    = "172.16.2.4"
+ wireguard_public_ip     = (known after apply)
```

Figure 21 — Terraform Apply — 44 Azure Resources Created

The entire infrastructure is deployed by a single script (Figure 22). `deploy-full.sh` does everything in order: Terraform creates 2 Proxmox VMs and 44 Azure resources, then generates WireGuard keys and runs 5 Ansible playbooks (PostgreSQL, security, WireGuard, exporters, replication). After that it builds 3 Docker images, pushes them to ACR, applies all K8s manifests, and waits for cert-manager to get a TLS certificate. Start to finish, about 16 minutes from an empty subscription.

Recap — Deployment Pipeline (`deploy-full.sh` ~ 16 min)

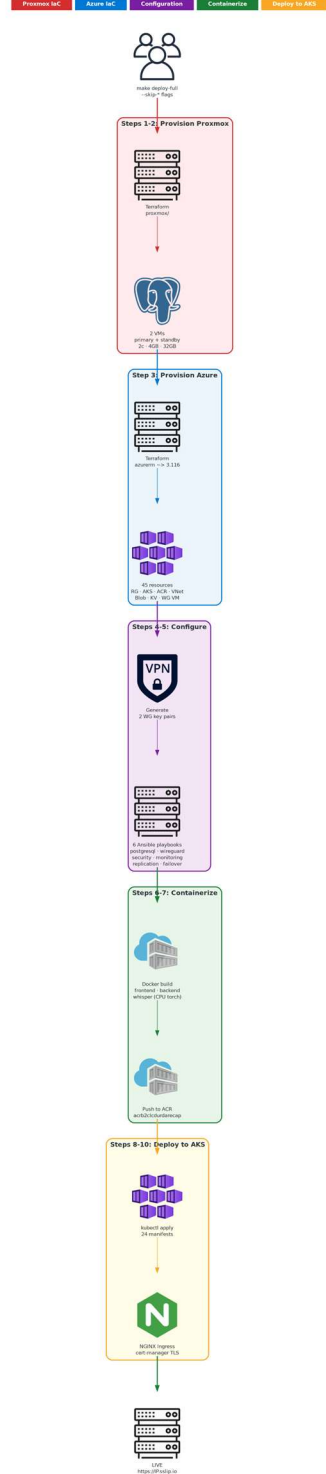


Figure 22 — Full Deployment Pipeline (Terraform → Ansible → Docker → K8s → Live)

3.6 Ansible (Configuration Management)

Where

ansible/ directory — 8 playbooks, 7 roles, inventory with db_primary, db_standby, and db_servers groups. Runs from david@david machine (on Proxmox network).

Why

Terraform creates VMs but does not configure them. Ansible handles everything inside the VM: installing PostgreSQL, hardening SSH, setting up the VPN tunnel, configuring replication, etc.

How

deploy-full.sh runs 5 playbooks in sequence. The other 3 are for specific tasks like VPN Gateway migration prep, failover setup, and manual disaster recovery.

Playbook	Target	Purpose
setup-postgresql.yml	db_primary + db_standby	PostgreSQL 16, users, SSL, replication slot, backups
security-hardening.yml	All VMs	UFW, fail2ban, SSH hardening, sysctl
configure-wireguard.yml	WG VM	WireGuard VPN tunnel + DNAT rules
configure-vpn.yml	DB servers	IPsec/strongSwan (prepared for future VPN Gateway migration)
setup-monitoring-exporters.yml	Proxmox VMs	node_exporter (9100) + postgres_exporter (9187)
setup-db-replication.yml	Primary + Standby	Streaming replication via pg_basebackup
setup-auto-failover.yml	Primary + Standby	repmgr + keepalived for automatic failover (VIP 10.0.96.250)
dr-failover.yml	Standby	Manual failover: promote standby via pg_ctlcluster promote

Figure 23 — Ansible Playbooks (8 total)

7 Ansible roles:

Role	Purpose
postgresql	Install PG 16, create DB/users, backup cron, replication scripts
common	SSH hardening, UFW, fail2ban, sysctl
wireguard	WireGuard peer configuration
node_exporter	Prometheus node metrics agent (port 9100)
postgres_exporter	PostgreSQL metrics agent (port 9187)
repmgr	PostgreSQL replication manager for automatic failover
keepalived	Virtual IP (10.0.96.250) management for transparent failover

Figure 24 — Ansible Roles (7 total)

```
PLAY RECAP *****
db-server          : ok=29   changed=24   unreachable=0   failed=0   skipped=0   rescued=0
  ignored=0
db-standby         : ok=7    changed=6   unreachable=0   failed=0   skipped=0   rescued=0
  ignored=0
```

Figure 25 — Ansible Playbook Run — Configuration Summary

3.7 Monitoring (Prometheus + Grafana)

Where

Dedicated monitoring namespace in AKS — Prometheus, Grafana, and Blackbox Exporter.

Why

Monitoring is a project requirement. I needed to cover the AKS cluster, the on-premise VMs, the VPN tunnel, and the application itself — basically everything that can break.

How

Prometheus has 14 scrape jobs (27 total targets): the backend API (9 custom metrics), whisper-worker (13 custom metrics), PostgreSQL via postgres_exporter, the Proxmox host via node_exporter, NGINX Ingress, the Kubernetes API, and blackbox probes. On-premise metrics come through the WireGuard DNAT.

I defined 22 custom metrics in total (all prefixed recap_). Prometheus scrapes them every 15 seconds, 5 alert rules fire on them, and about 50 Grafana panels visualize them across 6 dashboards.

Backend API — 9 custom metrics (backend/app/metrics.py):

Metric	Type	Labels	Description
recap_auth_attempts_total	Counter	method, result	Login/register attempts (local/azure, success/failure)
recap_recordings_created_total	Counter	input_type	Recordings created (upload/record/youtube/text)
recap_recordings_deleted_total	Counter	—	Total deletions
recap_file_upload_bytes	Histogram	—	Upload file size distribution (10KB–100MB buckets)
recap_gemini_requests_total	Counter	endpoint, result	Backend Gemini API calls — success/error/rate_limited
recap_gemini_duration_seconds	Histogram	endpoint	Gemini chat call latency (0.5s–60s buckets)
recap_gemini_daily_remaining	Gauge	—	Remaining daily Gemini quota (backend estimate)
recap_chat_messages_total	Counter	role	Chat messages by role (user/assistant)
recap_db_errors_total	Counter	—	Database connection/query errors

Figure 26 — Backend Custom Prometheus Metrics (9 total)

```
# backend/app/metrics.py – custom Prometheus metrics (excerpt)
auth_attempts_total = Counter("recap_auth_attempts_total", "...", ["method", "result"])
recordings_created_total = Counter("recap_recordings_created_total", "...", ["input_type"])
file_upload_bytes = Histogram("recap_file_upload_bytes", "...",
    buckets=[10_000, 100_000, 1_000_000, 5_000_000, 10_000_000, 50_000_000, 100_000_000])
gemini_requests_total = Counter("recap_gemini_requests_total", "...", ["endpoint", "result"])
gemini_daily_remaining = Gauge("recap_gemini_daily_remaining", "...")
```

Whisper Worker — 13 custom metrics (whisper-worker/worker.py):

Metric	Type	Labels	Description
recap_whisper_transcription_seconds	Histogram	—	Whisper transcription duration (1s–600s)
recap_whisper_transcriptions_total	Counter	result	Transcription outcomes (success/error/file_not_found)
recap_whisper_summarizations_total	Counter	result	Summarization outcomes (success/error/rate_limited)
recap_whisper_summarization_seconds	Histogram	—	Gemini summarization latency (0.5s–60s)
recap_queue_pending	Gauge	—	Recordings in pending state
recap_queue_transcribing	Gauge	—	Recordings currently being transcribed
recap_queue_summarizing	Gauge	—	Recordings currently being summarized
recap_queue_completed	Gauge	—	Total completed recordings
recap_queue_failed	Gauge	—	Total failed recordings

recap_stale_resets_total	Counter	phase	Stuck recordings auto-recovered
recap_worker_gemini_daily_count	Gauge	—	Gemini API calls made today
recap_worker_gemini_cooldown	Gauge	—	1 = Gemini in cooldown, 0 = available
recap_whisper_model_loaded	Gauge	—	1 when Whisper model is loaded and ready

Figure 27 — Whisper Worker Custom Prometheus Metrics (13 total)

The metrics are scattered across the backend routers (auth.py, recordings.py, chat.py), the gemini_service, and the whisper worker poll loop. On top of the 22 custom metrics, the backend also auto-instruments HTTP request metrics via prometheus_fastapi_instrumentator, and the on-premise exporters add standard pg_stat_* and node_* metrics.

5 Prometheus alert rules fire specifically on custom recap_* metrics:

Alert	Expression	Fires when
GeminiQuotaLow	(1000 - recap_worker_gemini_daily_count) < 50	<50 Gemini calls remaining today
GeminiQuotaExhausted	(1000 - recap_worker_gemini_daily_count) < 5	Summarization effectively stopped
QueueBacklog	recap_queue_pending > 10 for 5m	>10 recordings stuck pending
HighFailureRate	recap_queue_failed > 5	>5 failed recordings in queue
TranscriptionErrors	rate(recap_whisper_transcriptions_total{result="error"}[15m]) > 0 for 5m	Whisper errors occurring

Figure 28 — Alert Rules on Custom Metrics

Prometheus configuration defines scrape jobs for both cloud and on-premise targets:

```
# k8s/monitoring/prometheus-config.yaml - scrape config (excerpt)
scrape_configs:
- job_name: "recap-backend"
  static_configs:
    - targets: ["backend.recap-app.svc.cluster.local:8000"]
- job_name: "recap-whisper"
  static_configs:
    - targets: ["whisper-worker.recap-app.svc.cluster.local:8001"]
- job_name: "proxmox-node" # via WireGuard DNAT
  static_configs:
    - targets: ["172.16.2.4:9100"] # node_exporter
- job_name: "postgresql"
  static_configs:
    - targets: ["172.16.2.4:9187"] # postgres_exporter
```

Dashboard	Key Panels (~50 total)
Overview	Whisper model status, Gemini daily remaining, queue gauges (pending/transcribing/summarizing/completed/failed), stale resets, queue over time
API & HTTP	Auth attempt rate by method/result, successful vs failed logins, login failure ratio, registrations
AI Pipeline	Queue state + timeseries, transcription latency p50/p90/p99, Gemini calls remaining, request rate by source, Gemini latency p50/p95, recordings by input type, chat messages by role, upload size distribution
Infrastructure	CPU, memory, disk across AKS nodes
Database & Proxmox	Active connections, cache hit ratio, DB size, transaction rate, tuple ops, row counts, seq vs index scans, deadlocks
Network & Endpoints	VPN tunnel status, endpoint availability, cert expiry

Figure 29 — Grafana Dashboards (~50 panels across 6 dashboards)

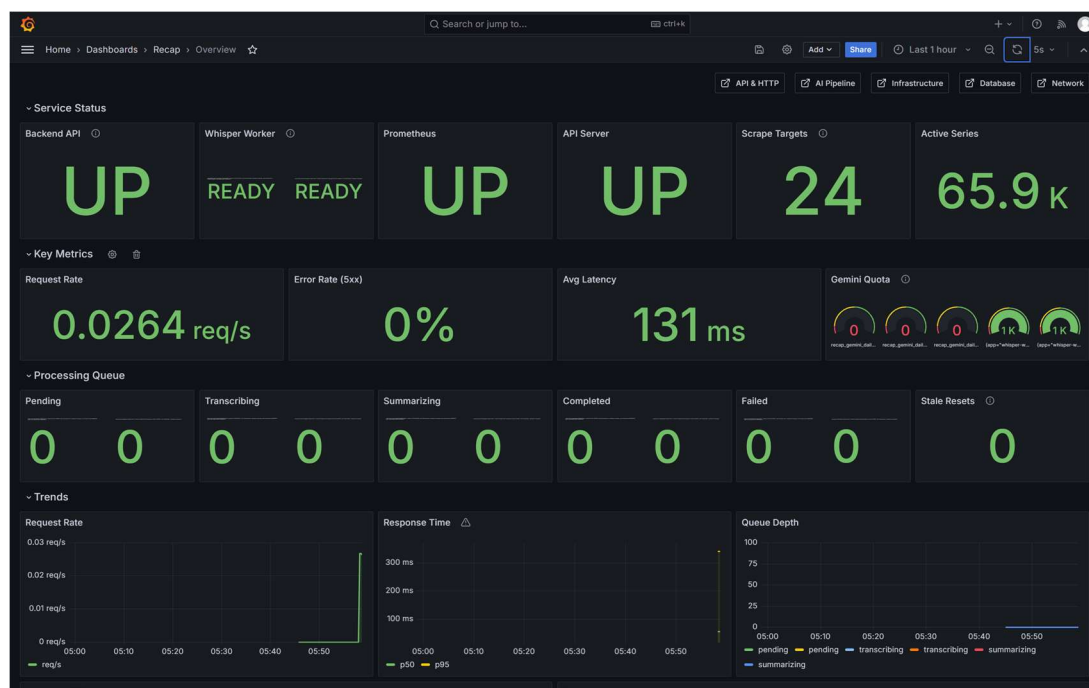


Figure 30 — Grafana — Overview Dashboard

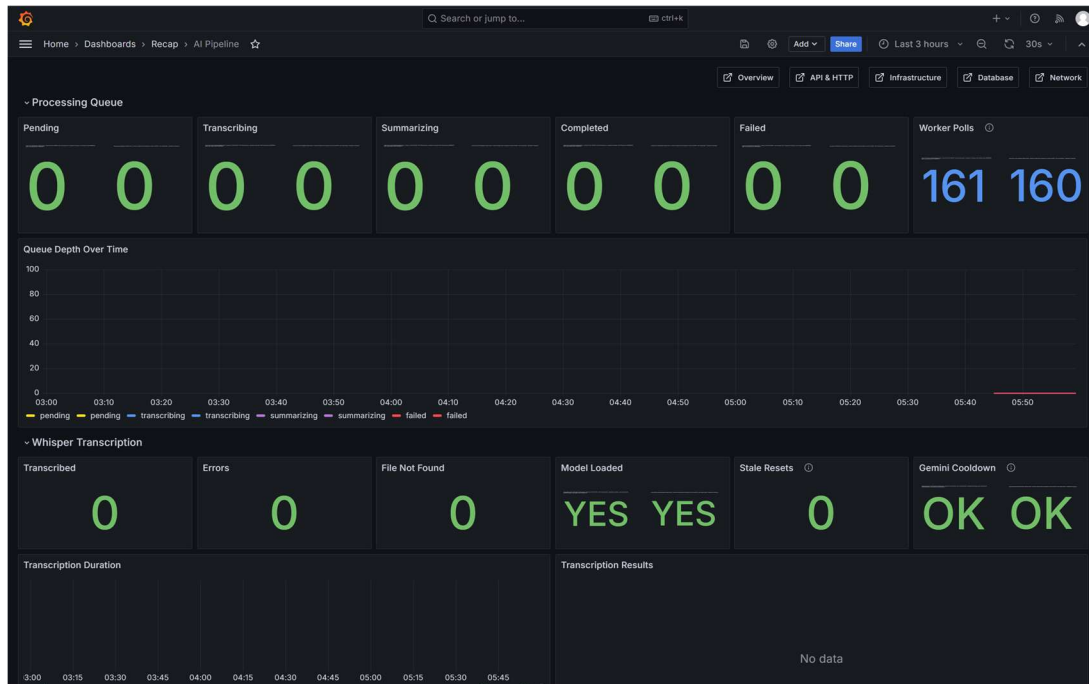


Figure 31 — Grafana — AI Pipeline Dashboard

Alert Group	Count	Examples
Service Health	4	BackendDown, WhisperWorkerDown, HighErrorRate
AI Pipeline	5	GeminiQuotaLow, QueueBacklog, TranscriptionErrors
Infrastructure	4	ContainerOOMRisk, CPUThrottled, PodRestarting
Database	7	PostgreSQLDown, ConnectionsHigh, ReplicationLag
Proxmox Host	5	ProxmoxNodeDown, DiskFull, HighMemory
Disaster Recovery	4	BackupTooOld, BackupMissing, ReplicationFailed
Network	3+	WireGuardTunnelDown, EndpointDown, SSLCertExpiring

Figure 32 — Prometheus Alert Rules (32 total across 7 groups)

The diagram below (Figure 33) ties everything together — how Prometheus, Grafana, and the Blackbox Exporter connect to the application, infrastructure, and on-premise targets described in the tables above.

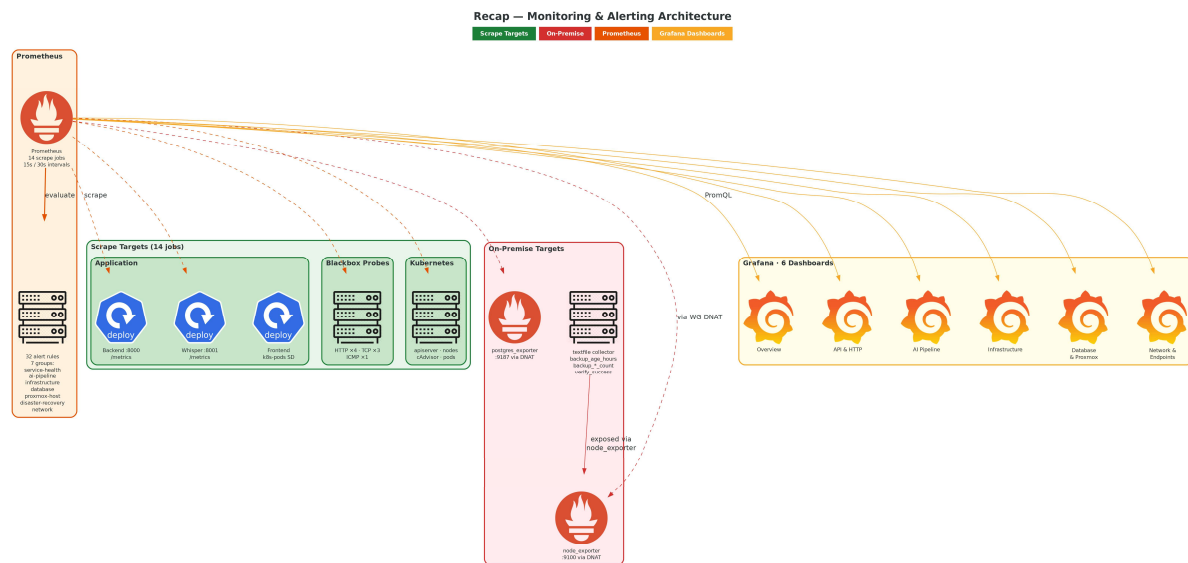


Figure 33 — Monitoring & Alerting Architecture

3.8 Naming Conventions & Resource Tagging

Naming Convention

All Azure resources follow the pattern {type}-b2clc-durda-recap, where b2clc is the class code and durda is my IAM code (teacher requirement). Resources that do not allow hyphens drop them.

Resource	Name	Pattern
Resource Group	rg-b2clc-durda-recap	{type}-b2clc-durda-recap
AKS Cluster	aks-b2clc-durda-recap	{type}-b2clc-durda-recap
Container Registry	acrb2clcdurdarecap	No hyphens allowed
Storage Account	stb2clcdurdarecap	No hyphens allowed
Key Vault	kv-b2clc-durda-recap	{type}-b2clc-durda-recap
Virtual Network	vnet-b2clc-durda-recap	{type}-b2clc-durda-recap
WireGuard VM	vm-b2clc-durda-wg	{type}-b2clc-durda-{role}
Managed Identity	id-b2clc-durda-recap	{type}-b2clc-durda-recap
NSG (general)	nsg-b2clc-durda-recap	{type}-b2clc-durda-recap
NSG (WireGuard)	nsg-b2clc-durda-wg	{type}-b2clc-durda-{role}
Route Table	rt-b2clc-durda-aks	{type}-b2clc-durda-{role}

Public IP (WG)	pip-b2clc-durda-wg	{type}-b2clc-durda-{role}
----------------	--------------------	---------------------------

Figure 34 — Azure Resource Naming Convention

Resource Tagging

Four mandatory tags on every resource, enforced by Azure Policy at the resource group level:

Tag	Value	Purpose
Team	BCLC24	Class identifier
Owner	DURDA	IAM code (uppercase, no numbers)
Project	Recap	Project name
Environment	prod	Deployment environment

Figure 35 — Mandatory Resource Tags

Tags are defined once in Terraform (locals.tf) and applied to every resource via local.common_tags. Azure audit shows 17 of 21 resources correctly tagged. The 4 untagged ones are in the MC_resource group that AKS creates and manages automatically — I scoped the tag policies to my resource group only, otherwise AKS cannot create its own internal resources.

Azure Policies

9 Azure Policy assignments enforce governance rules (4 at resource group level, 5 at subscription level):

Policy	Effect	What it enforces
Require Team tag	Deny	Blocks resource creation without Team tag
Require Owner tag	Deny	Blocks resource creation without Owner tag
Require Project tag	Deny	Blocks resource creation without Project tag
Require Environment tag	Deny	Blocks resource creation without Environment tag
Allowed VM SKUs	Deny	Only D2s_v4, D2as_v4, B2ats_v2, B2als_v2 allowed
Storage HTTPS only	Deny	Blocks storage accounts without HTTPS
Storage TLS 1.2	Deny	Blocks storage accounts below TLS 1.2
Key Vault soft-delete	Deny	Blocks Key Vaults without soft-delete enabled
AKS RBAC	Audit	Flags AKS clusters without RBAC enabled

Figure 36 — Azure Policy Assignments (9 total)

3.9 Operations Dashboard

Where

A separate web app in dashboard/ — Next.js 14 frontend (port 3100) + FastAPI backend (port 8100). It runs locally on my machine and talks to Azure CLI, kubectl, Terraform, and Ansible through subprocess calls. It is not deployed to AKS — it is a local ops tool.

Why

I came up with this idea late in the project, around the end of week 3. By that point I had already built the full infrastructure, and every time I needed to deploy, check pod status, run Ansible playbooks, or look at Prometheus targets, I was jumping between 5 different terminals and browser tabs. I really wanted a single place where I could see everything and trigger actions with one click. It was not in my original plan and it made me late on the report, but I think it was worth it — it shows the full scope of what the infrastructure can do from one screen.

How

The dashboard has 4 pages:

1. Overview — shows AKS cluster health (power state, Kubernetes version, node count), pod status across namespaces (Running/Total with color coding), and quick actions to start/stop the AKS cluster. Auto-refreshes every 15 seconds.
2. Pipeline — one-click full deployment via deploy-full.sh (~16 min), or run individual stages: Proxmox Terraform, Azure Terraform, Ansible (5 playbooks), Docker build+push, K8s apply. Each stage shows status (idle/running/success/failed) and duration. A WebSocket-based log terminal streams output in real time with color-coded lines (green for OK, red for errors, cyan for Ansible tasks). Also has destroy buttons for Azure, Proxmox, or both.
3. Resources — two tabs. Azure Resources lists everything in the resource group (name, type, location, tags). Terraform State shows managed resources and outputs for both Azure and Proxmox scopes, with buttons to run init, plan, or refresh.
4. Monitoring — connects to Prometheus via kubectl port-forward. Shows scrape targets (UP/DOWN with color coding), all alert rules sorted by state (firing first), key metrics (request rate, error rate, p95 latency), and a pod status grid for the recap-app namespace.

The backend is about 40 API endpoints covering Azure, Terraform, Ansible, Docker, Kubernetes, and Prometheus. Each action runs as an async subprocess with output captured and streamed over WebSocket. Job history (last 50 jobs) is persisted to a .state.json file.

Page	Key Features
Overview	AKS health, pod counts, node status, start/stop AKS, auto-refresh
Pipeline	One-click deploy, 5-stage pipeline, destroy, real-time log terminal
Resources	Azure resource list, Terraform state/outputs, init/plan/refresh
Monitoring	Prometheus targets, alert rules, PromQL queries, pod status grid

Figure 37 — Operations Dashboard Pages

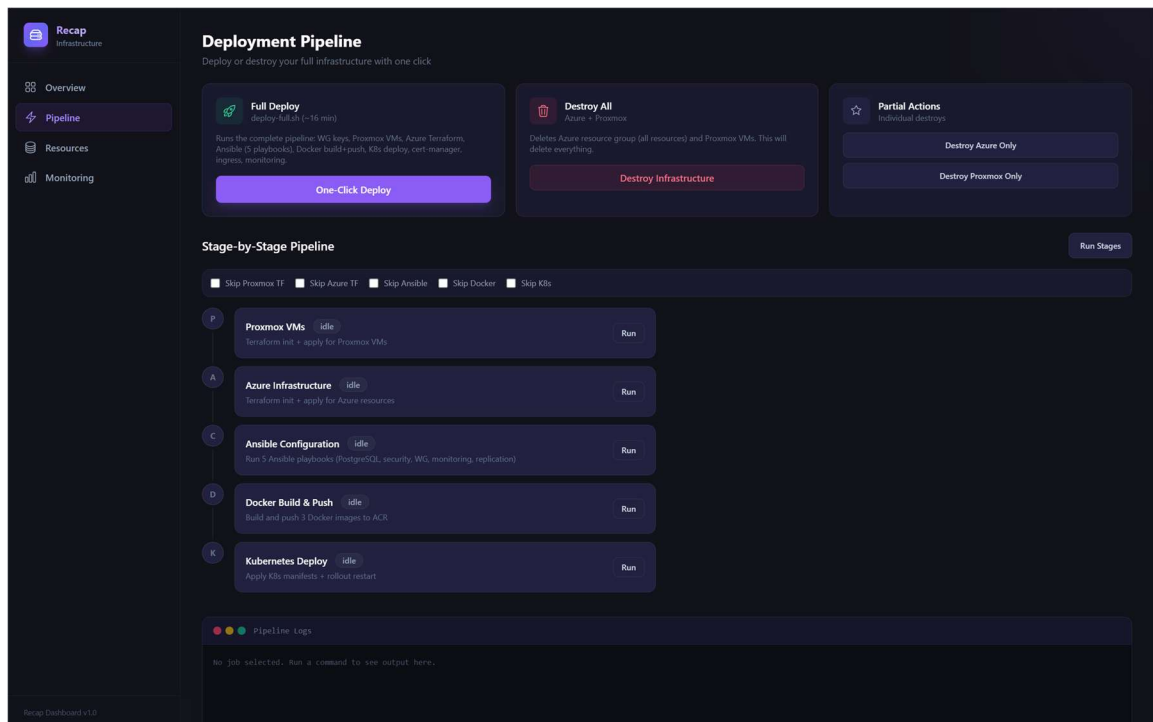


Figure 38 — Operations Dashboard — Overview

4. Major Challenges

4.1 AKS Resource Constraints — Fitting a Full Stack into 16 GB of RAM

The Problem

I need to run three workloads on AKS: a Next.js frontend, a FastAPI backend, and a Whisper transcription worker. With only \$100 in student credit, I had to pick the cheapest VM SKU that could actually handle the load.

My first idea was a single B2ats_v2 node (2 vCPUs, 1 GB RAM) — the cheapest burstable VM. That failed immediately: Whisper alone needs about 1 GB to run, leaving nothing for anything else. I tried D2as_v4 nodes (2 vCPUs, 8 GB each) next, but Azure kept failing to allocate them in swedencentral. D2s_v4 (Intel, same specs) worked. Two of those give me 4 vCPUs and 16 GB total, but Kubernetes system components eat about 4.3 GB, so I have roughly 11.7 GB for my app.

Solution 1: Whisper OOMKill — Raise Memory Limit to 1 Gi

Whisper was the biggest headache. I set the memory limit to 512 Mi because the model file is only 74 MB — seemed like plenty. But Kubernetes kept killing the pod (OOMKill). What I did not realize is that PyTorch loads the model into memory and then allocates extra buffers for inference. The actual runtime memory is about 1 GB, not 74 MB. I only figured this out by watching the Prometheus memory graphs and seeing the pod spike right before each kill. The fix was simple: raise the limit to 1 Gi and force CPU-only mode (`fp16=False`) so PyTorch does not try to allocate GPU memory:

```
# k8s/apps/whisper-deployment.yaml – resource limits
resources:
  requests:
    cpu: 250m
    memory: 512Mi
  limits:
    cpu: 500m
    memory: 1Gi # 512Mi caused OOMKill – base model needs ~1GB at runtime
```

The Dockerfile also needs to install CPU-only PyTorch before requirements.txt, otherwise pip pulls 3.3 GB of CUDA libraries and the Docker build runs out of disk space:

```
# whisper-worker/Dockerfile – CPU-only torch install
RUN pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu
RUN pip install --no-cache-dir --no-build-isolation -r requirements.txt
```

Solution 2: Soft Anti-Affinity for High Availability

With 2 replicas of frontend and backend on only 2 nodes, I needed anti-affinity rules to spread them out. I tried hard anti-affinity first, but it made pods unschedulable whenever one node was under memory pressure. Soft anti-affinity works better — it prefers spreading pods across nodes but lets them share a node if needed:

```
# k8s/apps/backend-deployment.yaml – soft anti-affinity
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - backend
          topologyKey: kubernetes.io/hostname
```

Normally each node runs one frontend and one backend pod. Under pressure, both can end up on the same node temporarily. I use the same pattern for whisper-worker and frontend.

Solution 3: Conservative HPA with Stabilization Windows

The backend HPA scales from 2 to 4 replicas at 70% CPU. I had to be careful here: scale up too fast and it starves the Whisper worker; too slow and users wait. Stabilization windows prevent the replica count from bouncing up and down:

```
# k8s/policies/hpa.yaml – backend autoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2
  maxReplicas: 4
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
  behavior:
```

```

scaleUp:
  stabilizationWindowSeconds: 60    # Wait 60s before adding a pod
  policies:
    - type: Pods
      value: 1
      periodSeconds: 60
scaleDown:
  stabilizationWindowSeconds: 300    # Wait 5min before removing a pod

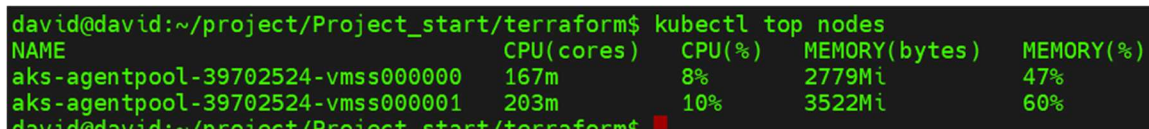
```

Quota Constraints — Unresolved

Azure for Students gives you 6 vCPUs in swedencentral. AKS uses 4 (2x D2s_v4) and the WireGuard VM uses 2 (B2ats_v2). That is the entire quota. I tried to add a second WireGuard VM for high availability, but there are literally zero 1-vCPU SKUs available in swedencentral (B1s does not exist there). So the WireGuard VM is a single point of failure, and there is nothing I can do about it without a quota increase.

Result

The cluster now runs stably: 5 application pods + monitoring across 2 nodes, using about 80% of available memory. Zero OOMKills since the 1 Gi fix. The HPA scales the backend to 4 pods under load without starving Whisper. A full destroy and redeploy takes ~16 minutes.



```

david@david:~/project/Project_start/terraform$ kubectl top nodes
NAME                                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
aks-agentpool-39702524-vmss000000  167m         8%       2779Mi          47%
aks-agentpool-39702524-vmss000001  203m         10%      3522Mi          60%
david@david:~/project/Project_start/terraform$

```

Figure 39 — kubectl top nodes — CPU/Memory per Node

Lessons Learned

1. A 74 MB model file does not mean 74 MB of RAM. Always measure actual runtime memory, not artifact size.
2. Hard anti-affinity sounds good on paper but breaks on small clusters. Soft anti-affinity is more practical.
3. Without Prometheus metrics for OOMKill and CPU throttling, I would still be guessing why pods kept dying.
4. Student subscriptions have hard vCPU quotas that force trade-offs. The WireGuard single point of failure is one of them.

4.2 Google Gemini Free Tier — Building a Reliable AI Pipeline on Unpredictable Infrastructure

The Problem

Gemini 2.5 Flash-Lite powers both summarization and the AI chat. I chose the free tier to keep costs at zero, but it comes with real limitations: 14 requests per minute, 1,000 per day, and 250,000 tokens per minute. On top of that, the free tier gives you no API logs or dashboard — there is no way to see what requests were made, what failed, or why. The Python SDK has no structured error codes, no system role for injecting context, and it regularly returns broken JSON. Every workaround below was

figured out through trial and error, because I had no other way to debug what was happening on Google's side.

Solution 1: Sliding Window Rate Limiter with Cooldown

Gemini calls come from two places: the whisper-worker (summarization) and the backend (chat). Each has its own sliding-window rate limiter capped at 14 RPM. When the daily quota runs out, the worker goes into a 1-hour cooldown:

```
# whisper-worker/worker.py – rate limiter (lines 118-160)
_call_timestamps: list[float] = []
_daily_call_count: int = 0
MAX_RPM = 14 # 1 under limit for safety
MAX_RPD = 1000
_gemini_cooldown_until: float = 0

def _wait_for_rate_limit():
    global _daily_call_count, _daily_reset_time
    now = time.time()
    if _daily_call_count >= MAX_RPD:
        _set_gemini_cooldown(3600) # 1-hour cooldown
        raise Exception("Gemini daily quota (1,000 RPD) exhausted")
    # Sliding window: drop timestamps older than 60s
    while _call_timestamps and _call_timestamps[0] < now - 60:
        _call_timestamps.pop(0)
    if len(_call_timestamps) >= MAX_RPM:
        sleep_time = 60 - (now - _call_timestamps[0]) + 0.5
        time.sleep(sleep_time)
    _call_timestamps.append(time.time())
    _daily_call_count += 1
```

The backend API (backend/app/services/gemini_service.py) uses the same pattern with an additional retry-with-backoff mechanism — on a 429 error, it sleeps 90 seconds and retries:

```
# backend/app/services/gemini_service.py – retry logic (lines 62-72)
def _call_with_retry(fn, max_retries: int = 2):
    for attempt in range(max_retries):
        try:
            return fn()
        except Exception as e:
            if _is_rate_limit_error(e) and attempt < max_retries - 1:
                time.sleep(90) # 90s backoff on 429
                _wait_for_rate_limit()
                continue
            raise
```

The obvious problem: these two counters do not talk to each other. If both fire at the same time, the combined rate can exceed 15 RPM. A shared counter (Redis or a DB semaphore) would fix this, but I did not want to add more infrastructure for an edge case.

Solution 2: String-Matching Error Detection

The Gemini SDK does not give you proper error codes. I have to check the exception message as a string to figure out what went wrong:

```
# whisper-worker/worker.py – error detection (lines 163-169)
def _is_rate_limit_error(e: Exception) -> bool:
    error_str = str(e).lower()
    return "429" in error_str or "resource" in error_str or "quota" in error_str

def _is_daily_quota_error(e: Exception) -> bool:
    return "perday" in str(e).lower() or "per_day" in str(e).lower()
```

Rate limit hit? Sleep 90 seconds. Daily quota gone? 1-hour cooldown. This is obviously fragile — if Google changes the error message wording, detection breaks silently. At least Prometheus tracks the cooldown state (`recap_worker_gemini_cooldown`) so I can see it in Grafana when something goes wrong.

Solution 3: JSON Fallback Parser

I ask Gemini to return JSON. It often ignores this and wraps the response in markdown code fences (````json ... ````), or just returns garbage. So I strip the fences and fall back to using the raw text as a summary if JSON parsing fails:

```
# whisper-worker/worker.py — JSON parsing with fallback (lines 348-363)
raw = response.text.strip()
# Strip markdown code fences that Gemini adds despite being told not to
if raw.startswith("```"):
    raw = raw.split("\n", 1)[1] if "\n" in raw else raw[3:]
    if raw.endswith("```"):
        raw = raw[:-3].strip()
try:
    result = json.loads(raw)
except json.JSONDecodeError:
    # Fallback: use raw text as summary if Gemini didn't return valid JSON
    result = {"summary": raw[:2000], "key_points": [], "action_items": []}
```

Users always get something, even when Gemini misbehaves. The downside is that fallback summaries are just raw text with no key points or action items.

Solution 4: Fake System Role for Chat Context

Unlike OpenAI, Gemini has no system role. My workaround is to fake it by injecting the transcript as a user/model exchange at the start of the chat history:

```
# backend/app/services/gemini_service.py — system role workaround (lines 94-96)
# Gemini has no system role — inject context as first user/model exchange
gemini_history = [
    {"role": "user", "parts": [system_context]},
    {"role": "model", "parts": ["Understood. I'll answer questions based on this recording."]}
]
```

This wastes tokens and sometimes the AI says things like "the instructions you gave me" in its responses, which is awkward. I also have to truncate context: 10,000 characters for chat, 15,000 for summarization. A 60-minute lecture produces about 30,000 characters of transcript, so the AI only sees the first half. The user is not told about this, which is a UX problem I have not fixed yet.

Solution 5: Stale Recording Recovery

When the daily quota runs out, recordings get stuck as "transcribing" or "summarizing" forever. The worker poll loop has a recovery mechanism that resets them:

```
# whisper-worker/worker.py — stale recovery (lines 397-443)
STALE_TRANSCRIBING_MINUTES = 10
STALE_SUMMARIZING_MINUTES = 15

def reset_stale_recordings(session):
    # Stuck transcribing > 10 min -> back to pending for retry
    session.execute(text("""
        UPDATE recordings SET status = 'pending', updated_at = NOW()
        WHERE status = 'transcribing'
        AND updated_at < NOW() - INTERVAL '10 minutes'
    ""
```

```

    """))
    # Stuck summarizing > 15 min -> re-queue
    session.execute(text("""
        UPDATE recordings SET updated_at = NOW()
        WHERE status = 'summarizing'
        AND updated_at < NOW() - INTERVAL '15 minutes'
    """))

```

Stuck recordings get re-queued automatically, but they will not actually process until the cooldown expires. A GeminiQuotaExhausted Grafana alert lets me know when this happens.

Result

With all these workarounds in place, the pipeline handles recordings reliably within the free-tier limits. The rate limiter avoids 429 errors in normal use, the fallback parser always produces some kind of summary, and the cooldown stops wasting calls after the daily quota is gone. Total API cost: \$0.

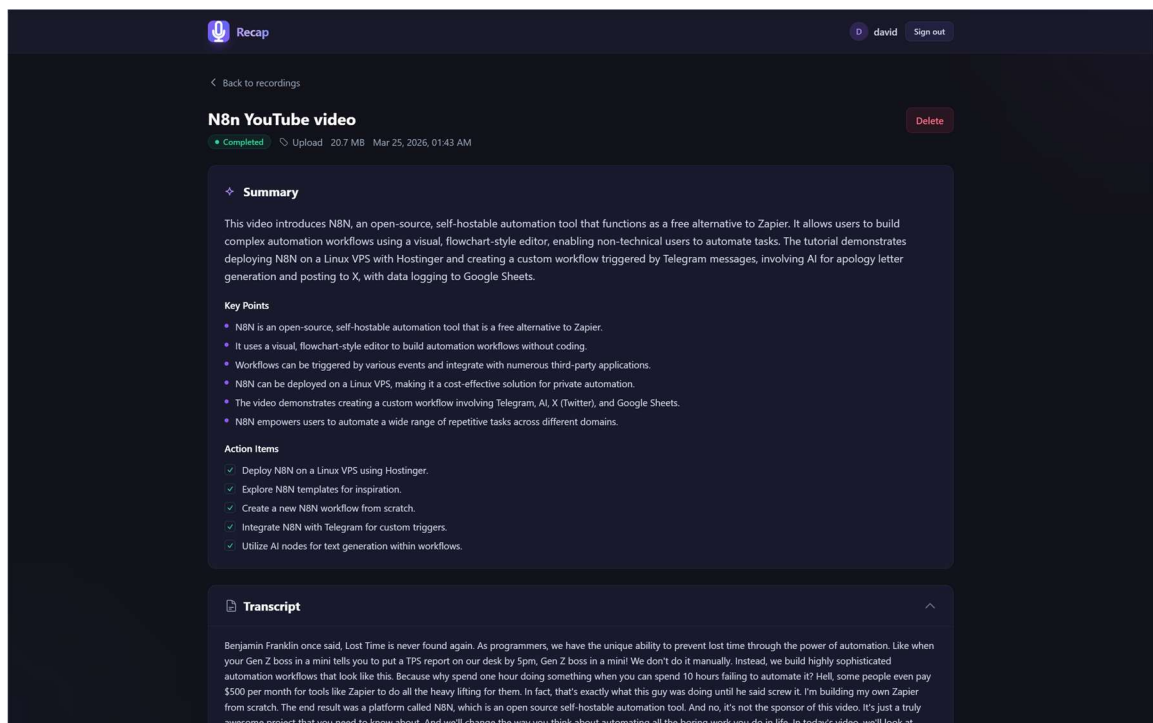


Figure 40 — Recording Detail — AI Summary, Key Points, Action Items

The AI chat works as a per-recording conversation (Figure 42). The user asks a question, the frontend POSTs to `/api/chat/:recording_id`, and the backend loads the transcript + chat history from PostgreSQL. It builds a Gemini prompt with the summary and transcript (capped at 10,000 chars) plus the conversation so far, sends it to Gemini, saves the response to `chat_messages`, and returns it. The same rate limiter and retry logic from summarization protect this path too.

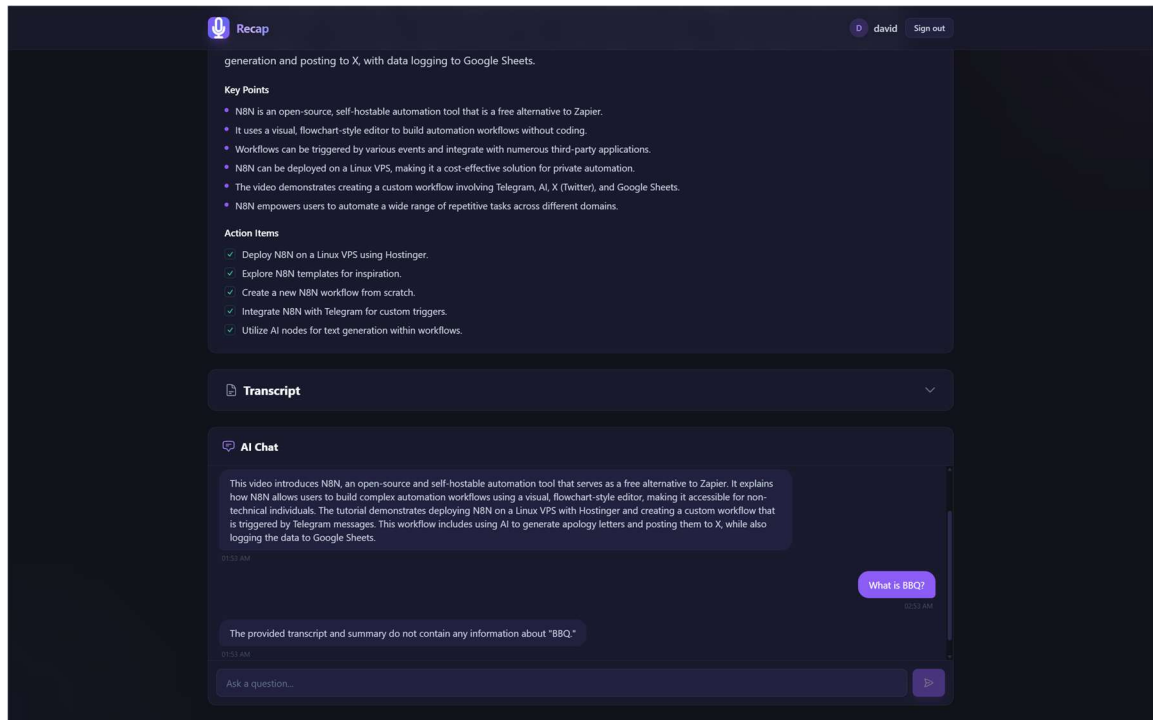


Figure 41 — AI Chat — Per-Recording Q&A

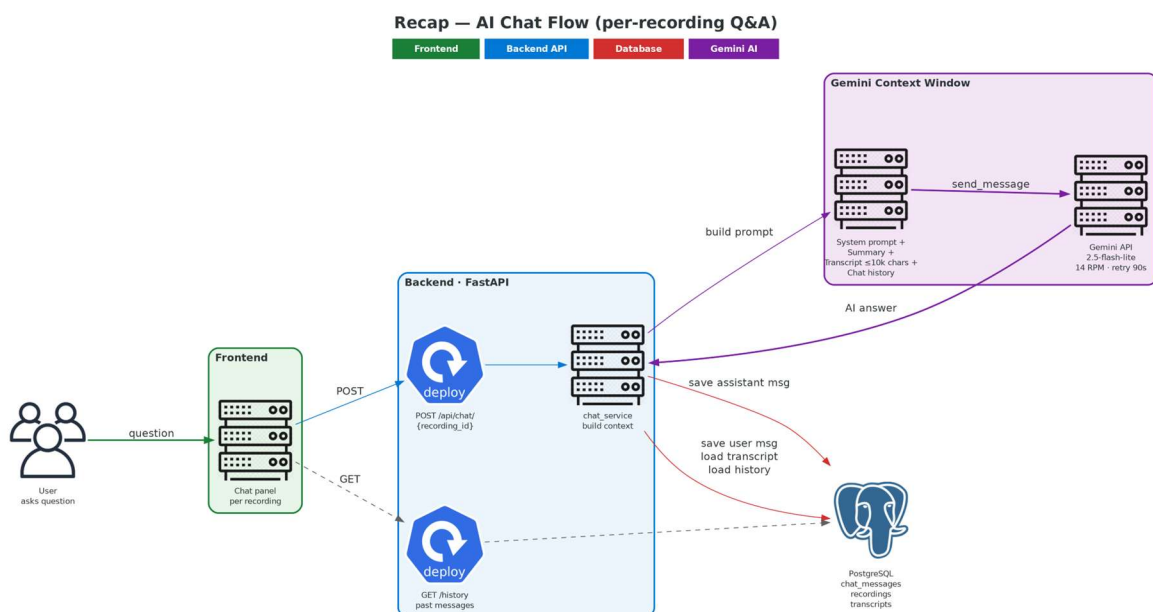


Figure 42 — AI Chat Flow (per-recording Q&A)

Lessons Learned

1. Free-tier APIs need defensive code everywhere: rate limiters, retries, cooldowns, fallback parsers. Every call can fail in a different way.
2. String-matching for error detection is ugly but sometimes it is all you have when the SDK does not give you proper error types.
3. Having two independent rate limiters for the same quota is a design flaw. A shared counter (Redis, DB) would be better.
4. Silently cutting a transcript in half is bad UX. Users should know when the AI only sees part of their recording.
5. Despite all these issues, Gemini's free tier let me ship AI features at zero cost. The trade-off was code complexity, not budget.

5. Security Concept

Security is organized in four layers, from edge to data (Figure 43). At the edge: NGINX Ingress with TLS from Let's Encrypt and security headers (CSP, HSTS, X-Frame-Options). Network layer: 20 Kubernetes NetworkPolicies (default-deny in both namespaces), NSG rules on the WireGuard VM, encrypted VPN. Application layer: rate limiting with slowapi, dual auth, non-root containers, Pydantic input validation. Data layer: PostgreSQL SSL-only, secrets in Key Vault (never in env vars), Blob Storage with soft-delete.

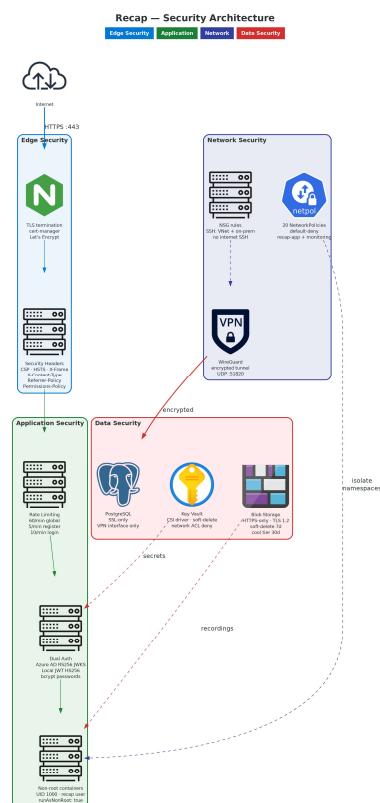


Figure 43 — Security Architecture (4 layers)

5.1 Cloud Security (Azure)

Layer	Security Measures
Identity (Azure AD)	Multi-tenant app, OAuth 2.0 + PKCE, JWT validation via JWKS, local auth with bcrypt (min 8 chars, upper+lower+digit)
AKS Cluster	RBAC enabled, 20 NetworkPolicies (default-deny), workload identity for Key Vault
Key Vault	Soft-delete enabled, CSI driver (secrets as files, never in env vars), RBAC access (Key Vault Secrets User role)
Blob Storage	Private access only, encryption at rest (SSE), TLS 1.2 enforced via policy, lifecycle to Cool after 30 days
Container Registry	Basic SKU with admin credentials, images stored in ACR
Azure Policies	9 assignments: 4 tag deny, allowed VM SKUs, storage HTTPS, TLS 1.2, KV soft-delete, AKS RBAC
TLS/HTTPS	cert-manager + Let's Encrypt production, NGINX Ingress ssl-redirect, HSTS header

Figure 44 — Cloud Security Measures

Name	Type	Status	Expiration date
azure-storage-connection-string		✓ Enabled	
database-url		✓ Enabled	
database-url-sync		✓ Enabled	
db-connection-string		✓ Enabled	
db-connection-string-sync		✓ Enabled	
gemini-api-key		✓ Enabled	
jwt-secret		✓ Enabled	
storage-connection-string		✓ Enabled	

Figure 45 — Azure Key Vault — Secrets

Name	Scope	Compliance state	Resource compliance	Non-compliant resources	Non-compliant policies
ASC Default (subscription: 286c0b0d-0352-4f21)	Azure for Students durda1	Non-compliant	11% (1 out of 9)	8	47
RBAC should be used on Kubernetes Services	Azure for Students durda1	Compliant	100% (1 out of 1)	0	0
Require a tag: Environment	Azure for Students durda1/rg-b2clc...	Compliant	100% (1 out of 1)	0	0
Require a tag: Owner	Azure for Students durda1/rg-b2clc...	Compliant	100% (1 out of 1)	0	0
Require a tag: Project	Azure for Students durda1/rg-b2clc...	Compliant	100% (1 out of 1)	0	0
Require a tag: Team	Azure for Students durda1/rg-b2clc...	Compliant	100% (1 out of 1)	0	0
Storage TLS 1.2 minimum	Azure for Students durda1	Compliant	100% (2 out of 2)	0	0
Secure transfer to storage accounts should be e	Azure for Students durda1	Compliant	100% (2 out of 2)	0	0
Allowed virtual machine size SKUs	Azure for Students durda1	Compliant	100% (1 out of 1)	0	0
Key vaults should have soft delete enabled	Azure for Students durda1	Compliant	100% (1 out of 1)	0	0
Allowed resource deployment regions	Azure for Students durda1	Compliant	100% (0 out of 0)	0	0

Figure 46 — Azure Policy — Compliance Dashboard

5.2 On-Premise Security (Proxmox)

Layer	Security Measures
SSH	Key-only auth (password disabled), hardened sshd_config
Firewall (UFW)	Default deny; allow SSH, PG from VPN (172.16.0.0/16), tunnel (10.99.0.0/24), replication (10.0.96.0/24)
Fail2ban	Active on SSH — bans IPs after repeated failed attempts
PostgreSQL	SSL-only, listen on VPN interface only, dedicated recap_user with limited privileges
WireGuard VPN	Curve25519 + ChaCha20-Poly1305, DNAT for ports 5432, 9100, 9187 only

Figure 47 — On-Premise Security Measures

5.3 Application Security

Layer	Security Measures
Rate Limiting	slowapi — 60/min default, 5/min register, 10/min login
Security Headers	CSP, HSTS, X-Frame-Options (DENY), X-Content-Type-Options, Referrer-Policy, Permissions-Policy
Non-root Containers	Backend + whisper-worker run as UID 1000 (recap user)
Input Validation	Pydantic schemas for all API inputs, 100 MB file limit, 60 min duration limit
Structured Logging	JSON via python-json-logger — no sensitive data in logs
Secrets Management	No secrets in code; Key Vault CSI or K8s secrets from deploy script

Figure 48 — Application Security Measures

I add security headers via a FastAPI middleware that runs on every response:

```
# backend/app/main.py — security headers middleware (lines 80-95)
@app.middleware("http")
async def security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    response.headers["Permissions-Policy"] = "camera=(), microphone=(self), geolocation=()"
    if request.url.scheme == "https":
        response.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
        response.headers["Content-Security-Policy"] = (
            "default-src 'self'; script-src 'self' 'unsafe-inline'; "
            "connect-src 'self' https://login.microsoftonline.com https://*.blob.core.windows.net; "
            "frame-ancestors 'none'"
        )
    return response
```

Rate limiting uses slowapi with a global default and stricter limits on auth endpoints:

```
# backend/app/main.py — rate limiting setup (line 36)
limiter = Limiter(key_func=get_remote_address, default_limits=["60/minute"])

# backend/app/routers/auth.py — auth-specific limits
```

```
@router.post("/api/auth/register", ...)
@limiter.limit("5/minute") # prevent brute-force registration
async def register(request: Request, ...): ...

@router.post("/api/auth/login", ...)
@limiter.limit("10/minute") # prevent credential stuffing
async def login(request: Request, ...): ...
```

5.4 Network Security

- 20 NetworkPolicy resources across 2 namespaces (recap-app: 10, monitoring: 10)
- Default-deny ingress and egress in both namespaces
- Backend: DNS + PostgreSQL (172.16.2.4:5432) + HTTPS (443) for Gemini API
- Frontend: DNS only (no direct external access)
- Whisper worker: DNS + PostgreSQL + HTTPS (Blob Storage)
- ACME solver pods: dedicated policies in both namespaces for cert-manager HTTP-01 challenges

Everything is blocked by default. Each pod gets only the exact ports it needs:

```
# k8s/policies/network-policies.yaml – zero-trust networking (excerpt)
# Default deny ALL ingress in recap-app namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: recap-app
spec:
  podSelector: {}
  policyTypes:
    - Ingress
---
# Frontend: accept traffic from NGINX Ingress only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-ingress
  namespace: recap-app
spec:
  podSelector:
    matchLabels:
      app: frontend
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: ingress-nginx
      ports:
        - protocol: TCP
          port: 3000
```

- NSG rules: nsg-b2clc-durda-recap allows HTTP/HTTPS; nsg-b2clc-durda-wg allows WireGuard (51820), SSH (VNet+on-prem only), PostgreSQL (AKS subnet only)

5.5 EU Compliance Note

Gemini API servers are probably not all in the EU, so transcript data sent for summarization might be processed outside European jurisdiction. For a real production deployment I would switch to something EU-hosted like Mistral AI (France) or run an open-source LLM on Proxmox. Everything else in the project is EU-based: Azure swedencentral (Sweden), Proxmox on-premise in Luxembourg.

6. Cost Usage

6.1 Azure Resource Costs

Resource	SKU	Monthly Cost	Notes
AKS Control Plane	Free tier	\$0	Free forever
AKS Nodes (2× D2s_v4)	Pay-as-you-go	~\$140/month	\$0.096/hr × 2 × 730 hrs
WireGuard VM (B2ats_v2)	Pay-as-you-go	~\$7/month	\$0.0094/hr × 730 hrs
Blob Storage (LRS Hot)	Standard	<\$1/month	Minimal usage
Azure Files (PVC)	Standard	<\$1/month	Shared upload storage
Key Vault	Standard	<\$1/month	Free operations
ACR	Basic	~\$5/month	3 repositories
Public IPs (×3)	Standard	~\$11/month	AKS outbound + NGINX LB + WG VM
Load Balancer	Standard	\$0 (included)	AKS managed — no separate LB charge
Azure AD	Free tier	\$0	<50K MAU
Total (running)		~\$160/month (~\$5.30/day)	
Total (stopped)		~\$15/month (~\$0.50/day)	AKS stopped, disks + IPs only

Figure 49 — Azure Resource Costs

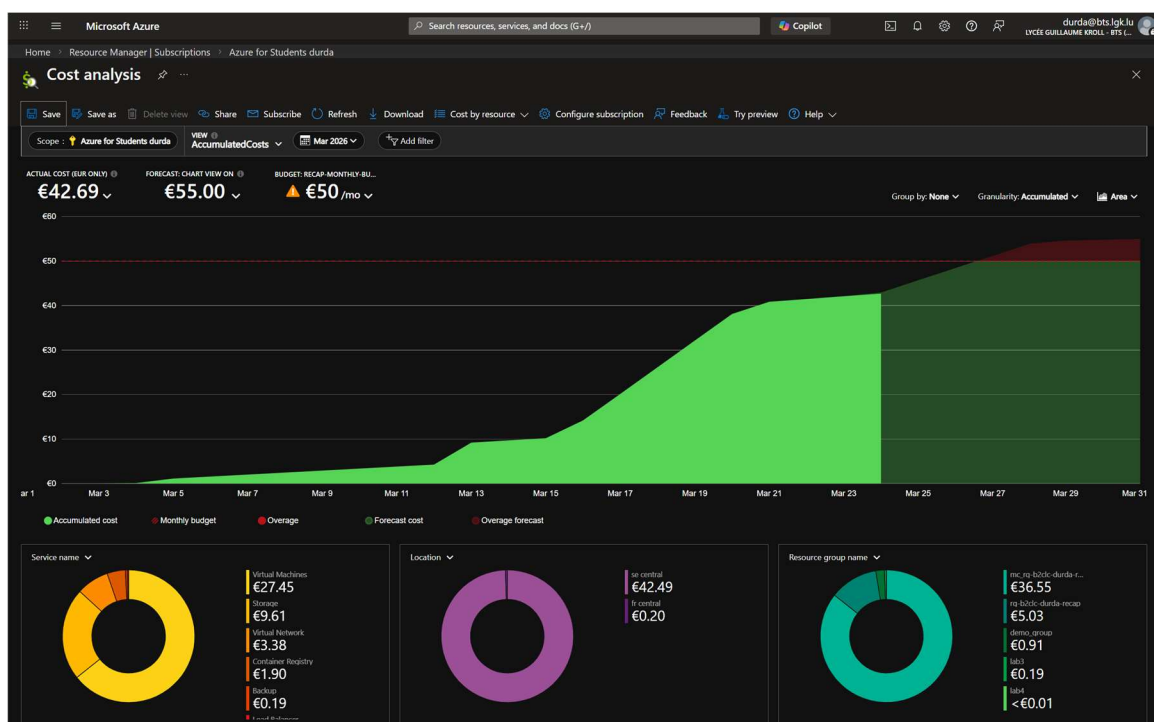


Figure 50 — Cost Analysis — Primary Subscription (durda)

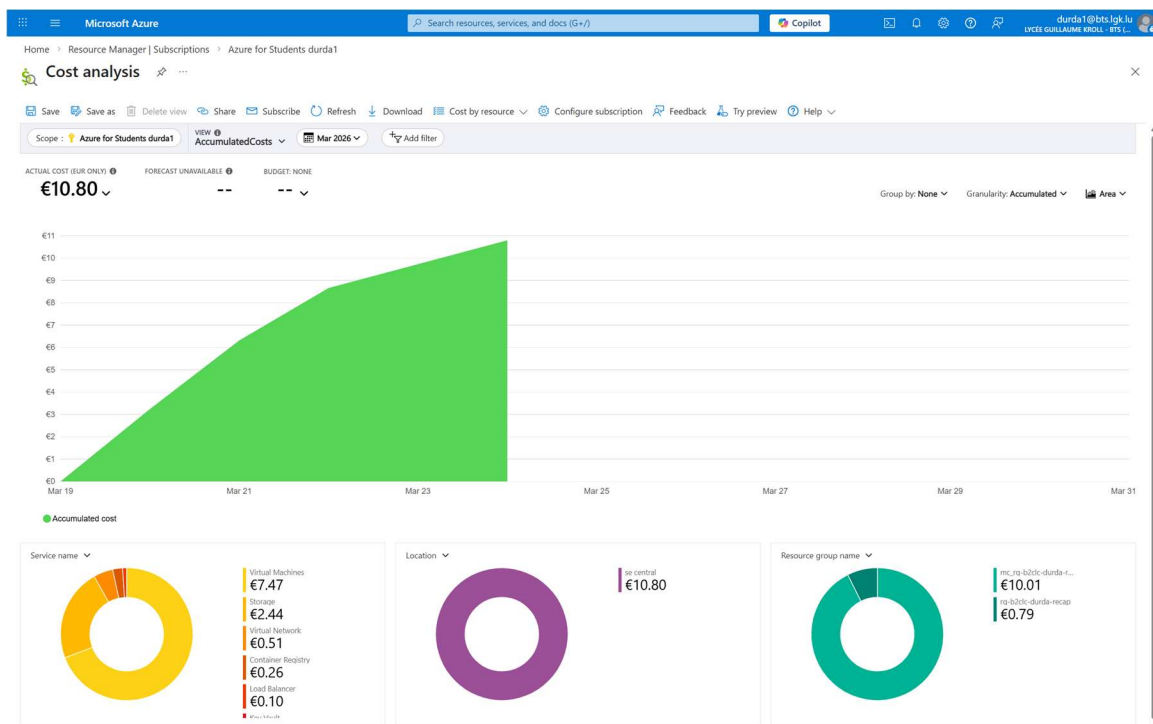


Figure 51 — Cost Analysis — Test Subscription (durda1)

6.2 Two-Subscription Strategy

The project used two separate Azure for Students subscriptions, each with \$100 credit:

Subscription	User	Purpose	Cost Strategy
Azure for Students durda	durda@bts.lgk.lu	Primary — production deployment, manual Portal setup, always-on for teacher testing	Higher spend; AKS kept running initially
Azure for Students durda1	durda1@bts.lgk.lu	Test — Terraform translation, IaC automation testing, destroy+redeploy cycles	Low spend; terraform destroy after each test

The primary subscription (durda@bts.lgk.lu) hosts the production deployment that teachers can access to test the application. Initially, the AKS cluster was left running continuously because the budget seemed sufficient. After feedback from Mr. Marc Schmit to keep an eye on costs, I started stopping the AKS cluster when not actively working — reducing the daily cost from ~\$6 to ~\$0.50. This subscription will remain online for the teachers to evaluate the live website during and after the presentation.

The second subscription (durda1@bts.lgk.lu) was used exclusively for testing the Terraform translation and automation. Infrastructure was created, validated, and immediately destroyed with terraform destroy after each test cycle. This kept the cost very low — each destroy+redeploy cycle runs for ~16 minutes and costs less than \$0.20. Multiple full end-to-end cycles (44 Azure resources + 2 Proxmox VMs + 5 Ansible playbooks) were validated on this subscription before the IaC was considered production-ready.

6.3 Cost Optimization Strategies

1. Database on Proxmox: Free on-premise VMs instead of Azure Database for PostgreSQL (~\$30+/month saved)
2. Stop AKS when not working: Reduces daily cost from ~\$6 to ~\$0.50 (adopted after instructor feedback)
3. WireGuard VM instead of VPN Gateway: B2ats_v2 ~\$7/month vs. VPN Gateway ~\$27+/month
4. Free-tier AI: Gemini 2.5 Flash-Lite (0 cost) instead of OpenAI API (\$0.002+ per call)
5. Self-hosted Whisper: CPU-only inference on AKS instead of paid transcription APIs
6. Blob lifecycle policy: Auto-tier to Cool storage after 30 days
7. CPU-only PyTorch: Reduces Docker image size by ~3 GB, faster builds
8. Separate test subscription: IaC testing on durda1 with terraform destroy keeps test costs near zero

6.4 Actual Spend

Primary subscription (durda@bts.lgk.lu): estimated ~\$70–90 out of \$100 over the 5-week project. Higher due to AKS running continuously in the first weeks before adopting stop/start discipline.

Test subscription (durda1@bts.lgk.lu): estimated ~\$10–15 out of \$100. Low due to terraform destroy after each test cycle — resources exist only for ~16 minutes per test.

7. Usage of AI

7.1 How AI Was Used

I used Claude Code (Anthropic) as a pair programming assistant throughout the project. It was involved in almost every part of the work:

1. Architecture: helped me evaluate VM SKUs, design the VPN topology, and plan the K8s deployment layout.
2. Code: generated FastAPI endpoints, React components, K8s manifests, Terraform configs, and Ansible playbooks. I reviewed and tested everything before deploying.
3. Debugging: helped me figure out OOMKill events, Terraform state conflicts, Ansible ordering bugs, and various Azure quirks (B2C deprecation, policy CLI failures).
4. Documentation: wrote CLAUDE.md context files, architecture docs, and the first draft of this report.
5. Iteration: every deploy cycle broke something new. I fed the logs to Claude and worked through the fixes together.

What made this more than just a chatbot was the tooling stack I built around it. Claude Code (Opus model) ran as a CLI agent with access to 6 MCP (Model Context Protocol) servers that gave it direct access to my infrastructure:

1. context7 — looked up library documentation on the fly (e.g. python-docx, FastAPI, Terraform provider schemas) so answers used current APIs, not outdated training data.
2. filesystem — read and wrote files across the project tree without me copy-pasting code into the chat.
3. git — inspected diffs, commit history, and branch state to understand what changed between deploy cycles.
4. sequential-thinking — broke complex multi-step problems (like the repmgr + keepalived failover design) into structured reasoning chains before generating code.
5. kubernetes — queried the live AKS cluster: pod status, logs, describe output, resource usage. When a pod crashed, Claude could read the logs directly instead of waiting for me to paste them.
6. postgres — ran read-only queries against the production database to verify schema, check replication status, and validate data after deployments.

This agent stack meant Claude was not just generating code in isolation — it could observe the actual state of the infrastructure, cross-reference documentation, and reason about multi-step problems before proposing changes. The combination of MCP servers turned a language model into something closer to a junior DevOps engineer with read access to everything.

7.2 AI Prompts

A selection of representative prompts is provided in Appendix A.

8. Personal Conclusion

8.1 Project Reflection

Looking back, I am genuinely proud of what I built. Recap started as a project proposal and ended up as a fully working SaaS application with hybrid infrastructure, automated deployment, monitoring, disaster recovery, and an AI pipeline — all running on a \$100 student budget. The fact that I can run a single script and go from an empty Azure subscription to a live application in 16 minutes still feels like a big achievement.

My planning was better than on any previous project. I mapped out the full architecture in week 1, broke the work into clear phases, and followed through without major surprises. The infrastructure came together smoothly — Terraform, Ansible, Kubernetes, the VPN tunnel, replication — each piece built on the last. I think that structured approach is the biggest thing I improved compared to my earlier projects.

That said, this is also the first time I missed a deadline, and I am not happy about that. What happened was that I came up with the operations dashboard idea on the last day. I knew it was late, but I really wanted it — a single web interface to manage the entire infrastructure felt like the kind of feature that ties the whole project together. So I put the documentation on hold and stayed up the entire night to get the dashboard working. It was the right call for the project, but it pushed the

report past the deadline. I should have either started the dashboard earlier or accepted that it would be a post-submission improvement.

If I could change one thing, it would be to plan all the features — including the “nice to have” ones — before writing a single line of code. The dashboard was not in my original scope, and adding it at the end created exactly the kind of time pressure I had avoided for three weeks. The lesson is simple: ambition is good, but it needs to be scoped from day one.

8.2 Project Management Reflection

I used ClickUp to track the project from week 1. I created user stories for every feature, logged problems as they came up, and used it to prioritize what to work on each day. For the first two and a half weeks it worked well — I could see exactly where I stood and what was left.

Then the last few days happened. As the deadline pressure built up and I started working on the dashboard, I stopped updating ClickUp. The tasks piled up unmarked, and by the end I was just coding without tracking anything. That defeats the purpose of project management tooling.

What I would change: define all features at the start, including stretch goals, so they are visible in the backlog from day one. When I added the dashboard late, it was not even in ClickUp — it was just an idea I chased. If it had been a backlog item from the beginning, I would have either planned time for it or consciously decided to cut it. That visibility is what project management tools are for, and I did not use it when it mattered most.

9. Planned Improvements

Between submission and the final presentation, I want to finish these:

- Add authentication to the operations dashboard (it is currently open)
- Finish the GitHub Actions CI/CD workflow so pushes to main auto-deploy
- Clean up the GitHub repo: branch protection, proper README, .gitignore review
- Update ClickUp with the final sprint status and burndown chart
- Polish the frontend: better error handling, loading states, responsive design
- Record demo videos showing the full user flow (upload → transcription → summary → chat)
- Run a proper security audit: CORS, input validation, EU data residency
- Improve Grafana dashboards: more useful panels, variables for filtering

Appendix A: AI Prompts

The following are representative prompts used during development with Claude Code (Anthropic):

Prompt 1 (Architecture): *"Design the AKS pod resource allocation for 2× D2s_v4 nodes running frontend, backend, and whisper-worker with monitoring"*

Prompt 2 (Debugging): *"Whisper worker keeps getting OOMKilled with 512Mi memory limit. The base model is 74 MB. Why does it need more?"*

Prompt 3 (IaC): *"Write a Terraform configuration for a WireGuard VM on Azure with cloud-init that auto-configures the tunnel and DNAT rules"*

Prompt 4 (Ansible): *"Create an Ansible playbook for PostgreSQL streaming replication between primary and standby VMs"*

Prompt 5 (Problem solving): *"Azure AD B2C is discontinued. What are the alternatives for user authentication that work with Azure for Students?"*

Prompt 6 (Optimization): *"The Whisper Docker image is 6 GB because it pulls CUDA libraries. How can I reduce it for CPU-only inference?"*

Prompt 7 (Security): *"Design NetworkPolicy resources for zero-trust networking in AKS with two namespaces"*

Prompt 8 (Deployment): *"Write a unified bash script that deploys the entire project from an empty Azure subscription to a running application"*

Table of Figures

Figure	Description	Location
Figure 1	Technology Stack	Section 1.5
Figure 2	Recap Dashboard — Recordings List	Section 1.5
Figure 3	Infrastructure Architecture Diagram	Section 2
Figure 4	Data Flow Diagram	Section 2.1
Figure 5	Network Layout	Section 2.2
Figure 6	Network Topology Diagram	Section 2.2
Figure 7	Project Folder Structure	Section 2.3
Figure 8	Azure Resources (rg-b2clc-durda-recap)	Section 2.4
Figure 9	Azure Portal — Resource Group Overview	Section 2.4
Figure 10	Azure AD App Registration — Recap	Section 3.1
Figure 11	Login Page — Microsoft + Email/Password Tabs	Section 3.1
Figure 12	Authentication Flow Diagram	Section 3.1
Figure 13	Proxmox — Database VMs	Section 3.2
Figure 14	Disaster Recovery & Automatic Failover Flow	Section 3.2
Figure 15	Database Schema (PostgreSQL 16)	Section 3.2
Figure 16	Azure Container Registry — 3 Docker Images	Section 3.3
Figure 17	Pod Resource Allocation	Section 3.4
Figure 18	Running Pods in recap-app Namespace	Section 3.4
Figure 19	Kubernetes Manifest Organization	Section 3.4
Figure 20	Terraform Configuration Files	Section 3.5
Figure 21	Terraform Apply — 44 Azure Resources	Section 3.5
Figure 22	Full Deployment Pipeline	Section 3.5
Figure 23	Ansible Playbooks	Section 3.6
Figure 24	Ansible Roles	Section 3.6
Figure 25	Ansible Playbook Run	Section 3.6

Figure 26	Backend Custom Prometheus Metrics (9)	Section 3.7
Figure 27	Whisper Worker Custom Prometheus Metrics (13)	Section 3.7
Figure 28	Alert Rules on Custom Metrics	Section 3.7
Figure 29	Grafana Dashboards (~50 panels)	Section 3.7
Figure 30	Grafana — Overview Dashboard	Section 3.7
Figure 31	Grafana — AI Pipeline Dashboard	Section 3.7
Figure 32	Prometheus Alert Rules (32 total)	Section 3.7
Figure 33	Monitoring & Alerting Architecture	Section 3.7
Figure 34	Azure Resource Naming Convention	Section 3.8
Figure 35	Mandatory Resource Tags	Section 3.8
Figure 36	Azure Policy Assignments (9 total)	Section 3.8
Figure 37	Operations Dashboard Pages	Section 3.9
Figure 38	Operations Dashboard — Overview	Section 3.9
Figure 39	kubectl top nodes — CPU/Memory	Section 4.1
Figure 40	Recording Detail — AI Summary	Section 4.2
Figure 41	AI Chat — Per-Recording Q&A	Section 4.2
Figure 42	AI Chat Flow (per-recording Q&A)	Section 4.2
Figure 43	Security Architecture (4 layers)	Section 5
Figure 44	Cloud Security Measures	Section 5.1
Figure 45	Azure Key Vault — Secrets	Section 5.1
Figure 46	Azure Policy — Compliance Dashboard	Section 5.1
Figure 47	On-Premise Security Measures	Section 5.2
Figure 48	Application Security Measures	Section 5.3
Figure 49	Azure Resource Costs	Section 6.1
Figure 50	Cost Analysis — Primary Subscription	Section 6.1
Figure 51	Cost Analysis — Test Subscription	Section 6.1